

Part IV: Using JavaScript and jQuery

Lesson 17. Introducing JavaScript

JavaScript is a programming language that's used to turn web pages into applications. JavaScript is used to manipulate the contents of a web page and to allow users to interact with web pages without reloading the page.

This is the first of three lessons in a row on JavaScript. In this lesson, I explain how JavaScript works and how to use it in your pages. In the next lesson, "[Using jQuery](#)," I discuss jQuery, a library that makes it much simpler to work with JavaScript, and then in the following lesson, "JavaScript Examples," I'll walk you through some real world examples. In this lesson, you learn about the basics of JavaScript by exploring the following topics:

- What JavaScript is
- Why you would want to use JavaScript
- The `<script>` tag
- An overview of the JavaScript language
- The browser as a programming environment
- Using JavaScript to handle browser events

Why Would You Want to Use JavaScript?

JavaScript changes the browser from an application for displaying documents to a platform for writing applications. It's useful because it enables the developer to manipulate the contents of a web page after it has loaded, making it possible to provide users with instant feedback when they make a change in a form, loading resources on demand for performance reasons, or building full-blown user interfaces that resemble desktop applications.

JavaScript is useful because it's deeply integrated with the browser. This integration allows programmers to manipulate various aspects of the browser behavior, as well as objects included on the page. JavaScript uses what's referred to as an *event-driven model* of execution. When you embed JavaScript code in a web page, it isn't run until the event it's associated with is triggered.

The types of events that can call JavaScript include loading the page, leaving the page, interacting with a form element in some way, clicking a link, or even just scrolling up or down. Plenty of other events are available, too. Often these events are utilized in what most users would consider to be annoying ways. For example, many sites open an additional window containing an advertisement when you navigate to one of their pages. This is accomplished using JavaScript and the page load event. Other sites open additional windows when you leave them; this is also accomplished using JavaScript triggered by the page unload event. Less annoying applications include validating forms before they are submitted or displaying extra information on a page when a user clicks a link without requiring a complete page refresh.

Note

This introduction will by necessity be briskly paced. There are many books written about JavaScript alone. The goal of these lessons is to introduce you to JavaScript, enable you to get started accomplishing tasks, and hopefully kindle your interest to dig into the language more deeply.

JavaScript enables you to manipulate web pages without sending a request back to the server or to send a request to the server to retrieve information without leaving the page that the user is on. Using these capabilities, you can change the contents of a page, change the style of elements on a page, validate user input before a user submits a form, and modify the behavior of the browser—all by using scripts embedded within your web pages. Let's look at some of the advantages of using JavaScript to enhance your web pages.

Ease of Use

JavaScript is a real programming language and is regularly used to build large, complex applications, including some you've probably seen, like Google Maps. At the same time, compared to many other programming languages, it's very easy to get started with JavaScript. You can add useful behavior to a web page with just a little bit of JavaScript added to the `onclick` attribute of a link or to a `script` tag at the top of an HTML document. And as you'll learn in [Lesson 18](#), jQuery makes it easy to add functionality to a page using just a few lines of code. The point is, don't be intimidated by JavaScript. You can start accomplishing things almost immediately.

Improving Performance

One of the main advantages of JavaScript is that it can provide user feedback instantly. Instead of requiring users to submit a form to see if their input was valid, you can let them know in real time. Not only can this improve user experience, but it can also make life easier for your server by preventing unnecessary form processing on the server. You can also use JavaScript to load resources when they're needed rather than when the page first loads. For example, you can load images that are not displayed immediately after the page loads to display the site more quickly. You can also use JavaScript to alter only the parts of the page that need to change when users interact with it. For example, you can process a search form using JavaScript and display the results on the same page so that a page reload isn't required.

Integration with the Browser

JavaScript enables you to manipulate objects on the page, such as links, images, and styles. You can also use JavaScript to control the browser itself by changing the size of the browser window, moving the browser window around the screen, and activating or deactivating elements of the interface. Technologies like Flash can provide an interactive interface, but they are not integrated into the browser in the same way that JavaScript is.

The <script> Tag

The <script> tag is used to include a JavaScript script in a web page, in much the same way that the <style> tag is used to add a style sheet to a page. The contents of the <script> tag are expected to be JavaScript source code. There are a couple of other ways to use JavaScript in your pages, too, but the <script> tag is a good place to start.

For the best results across all browsers, you should include the `type` attribute in the script tag, which specifies the type of content that the tag contains. For JavaScript, use `text/javascript`. HTML5 uses `text/javascript` as the default value for the `type` attribute, so if that is the type your script uses, you can leave it off.

The Structure of a JavaScript Script

When you include any JavaScript code in an HTML document (apart from using the <script> tag), you should also follow a few other conventions:

- HTML standards prior to HTML5 required that the <script> tag be placed between the <head> and </head> tags at the start of your document, not inside the <body> tag. However, for performance reasons, it's almost always a better idea to put your <script> tags at the bottom of the page. I'll discuss the reasons why later on.
- Unlike HTML, which uses the `<!-- comment tag -->`, comments inside JavaScript code use the `//` symbol at the start of a line. Any line of JavaScript code that starts with these characters will be treated as a comment and ignored.

Taking these three points into consideration, here's how the <script> tag is normally used:

[Click here to view code image](#)

```
<html>
<head>
<title>Test script</title>
</head>
<body>
  Your Web content goes here
<script>
  // Your JavaScript code goes here
</script>
</body>
</html>
```

You can place your <script> tag in either the head or the body of your document. But because JavaScript forces the browser to load it as a single thread, it's best to place it at the bottom of your pages, right before the closing </body> tag. This ensures that your pages load as quickly as possible.

The `src` Attribute

Besides the `language` attribute, the `<script>` tag can also include an `src` attribute, which allows a JavaScript script stored in a separate file to be included as part of the current web page. This feature enables you to share JavaScript code across an entire website, just as you can share a single style sheet among many pages.

When used this way, the `<script>` tag takes the following form:

[Click here to view code image](#)

```
<script src="http://www.example.com/script.js">
```

The `src` attribute will accept any valid URL, on the same server or on another. Naming your JavaScript files with the extension `.js` is customary.

JavaScript and the Chrome Development Tools

You've already seen how useful the Chrome Development Tools are when it comes to inspecting the source code of a page and diagnosing CSS issues on a web page. Developer tools like these were originally introduced to help people who were writing JavaScript. There are two incredibly useful features for JavaScript developers in the Developer Tools. The first is the JavaScript Debugger, which I'll discuss a bit later on. The second is the JavaScript Console, which provides a place to enter JavaScript code, execute it, and view the results interactively.

As I walk you through the JavaScript language, you can use the Console of the Chrome Development Tools (or the JavaScript Console in whichever browser you prefer) to run the examples. JavaScript errors that occur as you interact with the page are also printed in the Console, so when you're adding JavaScript to your own pages, you'll use it to investigate the bugs that inevitably crop up as you write your own JavaScript. You can see what the Console looks like in [Figure 17.1](#).

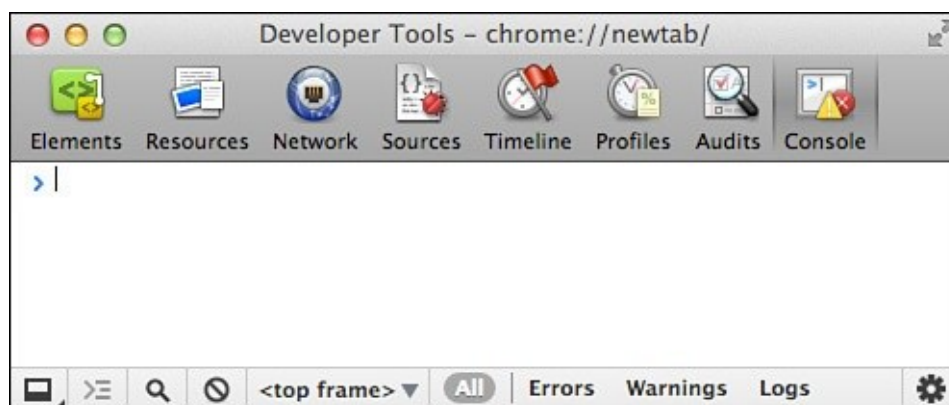


FIGURE 17.1 The Chrome Developer Tools JavaScript Console.

To view the Console, you open the Developer Tools and click on the Console tab. The `>` prompt is where you enter your JavaScript code to be evaluated. The Console is what's known as a REPL, which is short for Read-eval-print Loop. That's because it reads the code you enter, evaluates it, and then prints the results. It's a place you can experiment more easily than editing the JavaScript on a page, loading it in a browser, and then viewing the results.

To see how it works, open the Console and type “**1 + 1**” at the prompt. On the next line, you’ll see the number 2, the results of that mathematical expression. That expression happens to be valid JavaScript, and the Console automatically evaluates it and prints the results for you. Now try this:

```
alert(1 + 1);
```

You can see what the results look like in [Figure 17.2](#).

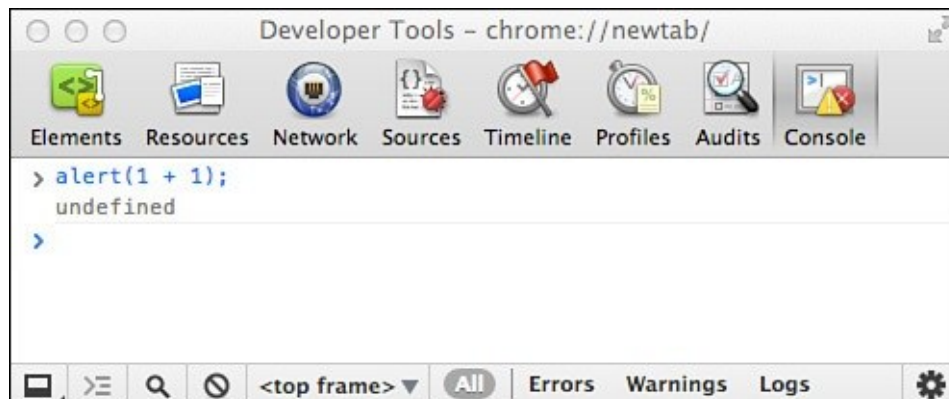


FIGURE 17.2 Expressions evaluated using the Console.

When you enter that line, an alert dialog will pop up containing the number 2. As you can see, you can affect the browser environment using the Console. The word *undefined* is printed in the Console. `alert`, which I’ll discuss further later, opens the dialog box. It is also an expression, just like `1 + 1`, and the results of evaluating that expression are printed in the Console window. In this case, the expression’s result is *undefined*, which is why that’s printed. That’s what any expression that does not return a useful value returns in JavaScript. If you’re not already a programmer, you may be wondering what an expression is at all. I’ll get into that a bit later as well.

The JavaScript Language

When JavaScript is included in a page, the browser executes that JavaScript as soon as it is read. Here’s a simple page that includes a script; the output is included in [Figure 17.3](#):

Input ▼

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
  <title>A Simple JavaScript Example</title>
</head>
<body>

  <p>Printed before JavaScript is run.</p>

  <p>
    <script>
      document.write("Printed by JavaScript.");
    </script>
  </p>
```

```
<p>Printed after JavaScript is run.</p>
```

```
</body>
```

```
</html>
```

Output ▼



FIGURE 17.3 The results of a simple script.

The page includes a single line of JavaScript, which writes the “Printed by JavaScript” on the page. The text is printed between the other two paragraphs on the page, demonstrating that the browser executed the JavaScript as soon as it got to it.

Before I start talking about what this code does, let me talk for just a moment about its structure. JavaScript programs are made up of individual statements, which are terminated by a semicolon. This program consists of a single statement or line of code.

As you can see, this line adds text to the page. Let me break down what this does, exactly. First, you might notice that `document.write()` and `window.alert()`, which I use earlier, look similar. Both `window` and `document` are things. `write` and `alert` are messages that you can send to those things. In both cases, you need content for those messages. In the latter example, the content of the `write` message is “Printed by JavaScript.”

In programming, the term used for things is *objects*, and the term used for the messages is *methods*. The content of the message is said to be passed to the method and is referred to as the method’s *argument*. So in programmer jargon, I’d say that I called the `write` method of the `document` object with the argument “Printed by JavaScript.” The `document` and `window` objects are provided by the browser—the environment in which JavaScript runs. You can also declare objects of your own; they’ll be discussed further along.

The `document` object is a representation of the current page that is accessible by JavaScript. Whenever you manipulate the page using JavaScript, you do so by calling methods of the `document` object.

The bit of text that I passed to the `document.write()` method in the previous

example is called a string in the vocabulary of programming. The `document.write()` method expects a string argument, which it then adds to the source of the document. Some programming languages are strict about data types, so if a method expects you to give it a string as an argument and you give it a number instead, an error will occur. JavaScript isn't like that. If you pass a string to the `document.write()` method, it will print it out unchanged. If you pass it some other kind of thing, it will do its best to convert it to a string and print it out.

So, for example, you can give it a number:

```
<script>
    document.write(500);
</script>
```

It will convert the number to a string and print it on the page. Or you can even pass it an object, like this:

[Click here to view code image](#)

```
<script>
    document.write(document);
</script>
```

The results are in [Figure 17.4](#).

Output ▼

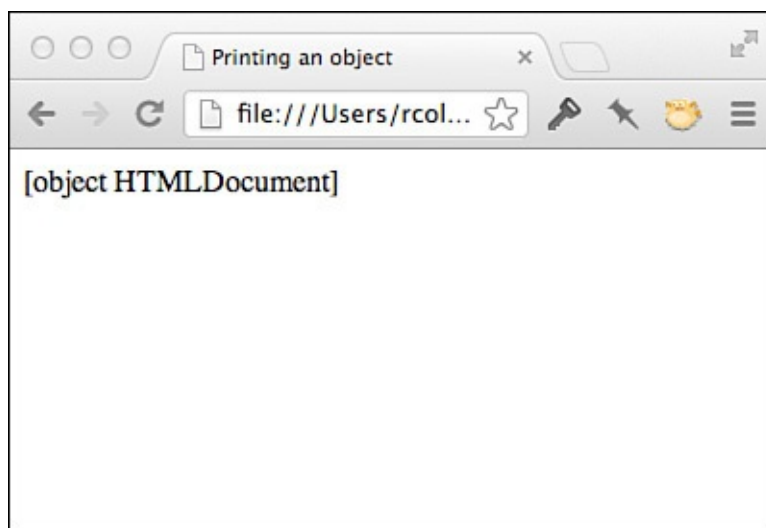


FIGURE 17.4 Attempting to write an object to the page.

That's the string representation of the `document` object. Many programming languages would print an error if you tried to use an object like `document` with a method that accepts a string. Not JavaScript. It makes do with what you give it.

Operators and Expressions

In my introduction to the JavaScript Console in the Chrome Web Developer Tools, I talked a bit about expressions. An *expression* is a snippet of code that can be evaluated to return a result. You may recognize the term expression from math, and indeed, many expressions are mathematical expressions. Here's a simple mathematical expression that you can enter in the Console:

In this case, JavaScript will multiply 10 by 50. That's an expression. There are also string expressions. For example, you can use the + operator to join strings together, like this:

[Click here to view code image](#)

```
"The bird a nest," + " the spider a web," + " man friendship."
```

Expressions are built using operators. You've already seen a couple: * for multiplication and + for combining strings or adding numbers. [Table 17.1](#) lists more operators provided by JavaScript, along with examples. (For a full list of all the supported operators, refer to the online JavaScript documentation.)

Operator	Example	Description
+	5 + 5	Adds the two numeric values; the result is 10.
+	"Java" + "Script"	Combines the two string values; the result is JavaScript.
-	10 - 5	Subtracts the second value from the first; the result is 5.
*	5 * 5	Multiplies the two values; the result is 25.
/	25 / 5	Divides the value on the left by the value on the right; the result is 5.
%	26 % 5	Obtains the modulus of 26 when it's divided by 5. (Note: A <i>modulus</i> is a function that returns the remainder.) The result is 1.

TABLE 17.1 JavaScript Operators and Expressions

There are many other operators, too. All the examples here used literal values in the expressions, but there are other options as well. You can use values returned by methods in expressions if you choose, as in the following example:

```
Math.sqrt(25) - Math.sqrt(16)
```

This example uses the `Math` object, another object built in to JavaScript. It provides a number of methods that perform a variety of mathematical operations so that you don't have to write the code to perform them yourself. `Math.sqrt()` is a method that returns the square root of a number. In this case, I subtracted the square root of 16 from the square root of 25. You can enter any of these expressions in the Console and see the results.

Variables

Thus far, I've been manipulating values and printing them directly to the page. The next fundamental building block of writing scripts is temporary storage of those values so that they can be reused. Like all programming languages, JavaScript supports the use of variables. A *variable* is a user-defined container that can hold a number, text, or an object. Creating variables and retrieving their values is simple in the Console, as shown in [Figure 17.5](#).

Input ▼

[Click here to view code image](#)

```
var message = "Message"
message
```

Output ▼

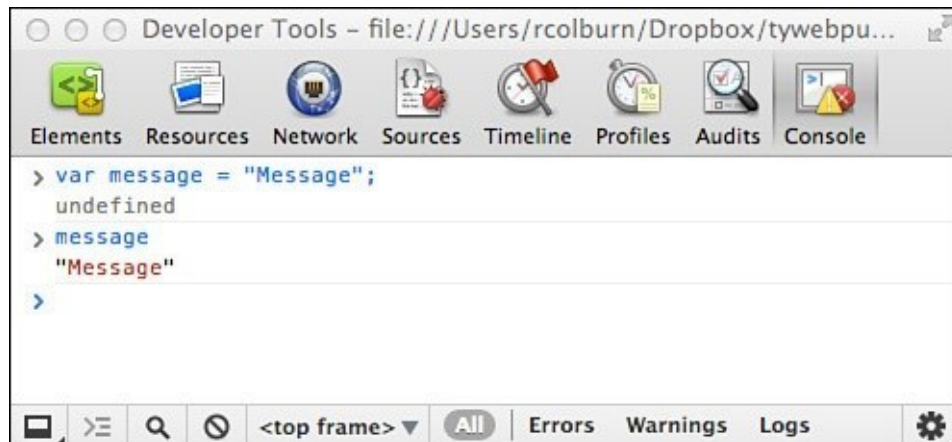


FIGURE 17.5 A variable declaration in the console.

In that example, I created a variable called `message` and then printed its value by entering it as an expression in the Console. You can also assign the results of an expression to a variable:

```
var sum = 5 + 5;
```

And you can use variables in your expressions:

[Click here to view code image](#)

```
var firstName = "George";
var lastName = "Washington";
var name = firstName + " " + lastName;
```

Let's break down a variable declaration into pieces. Here's a declaration:

```
var message = "My message";
```

The line begins with `var`, which indicates that this is a variable declaration. The name of this variable is `message`. There are a number of rules that apply to naming variables, which I will list shortly. The assignment operator (`=`) is used to assign a value to the variable when it's declared. The value returned by the expression on the right side of the operator is assigned to the newly declared variable.

Variable names must conform to the following rules:

- Variable names can include only letters, numbers, and the underscore (`_`) or dollar sign (`$`) character.
- Variable names cannot start with a number.
- You cannot use reserved words as a variable name. Reserved words are words that have a specific meaning for the JavaScript interpreter. For example, naming a variable `var` won't work. [Table 17.2](#) contains a full list of JavaScript reserved words.

<code>abstract</code>	<code>final</code>	<code>return</code>
<code>arguments</code>	<code>finally</code>	<code>short</code>
<code>boolean</code>	<code>float</code>	<code>static</code>
<code>break</code>	<code>for</code>	<code>super</code>
<code>byte</code>	<code>function</code>	<code>switch</code>
<code>case</code>	<code>goto</code>	<code>synchronized</code>
<code>catch</code>	<code>if</code>	<code>this</code>
<code>char</code>	<code>implements</code>	<code>throw</code>
<code>class</code>	<code>import</code>	<code>throws</code>
<code>const</code>	<code>in</code>	<code>transient</code>
<code>continue</code>	<code>instanceof</code>	<code>true</code>
<code>debugger</code>	<code>int</code>	<code>try</code>
<code>default</code>	<code>interface</code>	<code>typeof</code>
<code>delete</code>	<code>let</code>	<code>var</code>
<code>do</code>	<code>long</code>	<code>void</code>
<code>double</code>	<code>native</code>	<code>volatile</code>
<code>else</code>	<code>new</code>	<code>while</code>
<code>enum</code>	<code>null</code>	<code>with</code>
<code>eval</code>	<code>package</code>	<code>yield</code>
<code>export</code>	<code>private</code>	
<code>extends</code>	<code>protected</code>	
<code>false</code>	<code>public</code>	

TABLE 17.2 JavaScript Reserved Words

- As a matter of style, JavaScript variables begin with a lowercase letter. If a variable name contains multiple words, usually an underscore is used to join the two words, or the first letter of the second word is uppercase. So you would write `my_variable` or `myVariable`.

Not all the reserved words in [Table 17.2](#) are currently used in JavaScript; some have been placed off limits because they might be added to the language in the future.

Here are a couple of additional notes on variable assignment. You don't have to assign a value to a variable when you declare it. You can declare the variable without an assignment so that it can be used later. For example:

```
var myVariable;
```

If you entered `myVariable` in the console as an expression, it would be evaluated to `undefined` because nothing has been assigned to it. However, if you enter the name of an as yet undeclared variable (like `myNewVariable`), you'll see an error message.

You can also assign new values to variables after they've been declared, as follows:

```
myVariable = "My value";
```

Control Structures

To get your scripts to actually do something, you'll need control structures, which come in two main varieties. There are conditional statements, which are used to make decisions, and loops, which enable you to repeat the same statements more than once.

The `if` Statement

The main conditional statement you'll use is the `if` statement. The statements inside an `if` statement are executed only if the condition in the `if` statement is true. If you were writing code in English rather than JavaScript, an `if` statement would read like this: "If the background of this element is blue, turn it red." There's also an `else` clause associated with `if`. The statements in the `else` clause are executed if the `if` statement's condition is false. An `if` statement with an `else` clause reads like this: "If the value of this variable is blue, change it to red; otherwise, change it to blue."

Let's look at a simple example, which will work in the Console:

```
var color = "red";
if (color == "blue") {
    color = "red";
} else {
    color = "blue";
}
color;
```

In this example, I've created a variable named `color` and use that in my `if` statement. Later, I'll explain how to retrieve information from the page, style sheets, and form elements in your JavaScript code and make changes to them. For now, it's easier to explain the conditional statements with hard-coded values. The statement begins with the `if` keyword, followed by the condition enclosed within parentheses. The statements to be executed if the condition is true are placed within curly braces. In this case, I've also included an `else` clause. The statement associated with it is also enclosed in curly braces. Finally, let's look at the condition. It is true if the variable `color` is equal to the value "blue". In this case, the condition is false, so the `else` clause will be executed.

The `==` operator tests for equality and is but one of several conditional operators available in JavaScript. [Table 17.3](#) contains all the conditional operators.

Operator	Operator Description	Notes
<code>==</code>	Equal to	<code>a == b</code> tests to see whether <code>a</code> equals <code>b</code> .
<code>!=</code>	Not equal to	<code>a != b</code> tests to see whether <code>a</code> does not equal <code>b</code> .
<code><</code>	Less than	<code>a < b</code> tests to see whether <code>a</code> is less than <code>b</code> .
<code><=</code>	Less than or equal to	<code>-a <= b</code> tests to see whether <code>a</code> is less than or equal to <code>b</code> .
<code>>=</code>	Greater than or equal to	<code>-a >= b</code> tests to see whether <code>a</code> is greater than or equal to <code>b</code> .
<code>></code>	Greater than	<code>a > b</code> tests to see whether <code>a</code> is greater than <code>b</code> .

TABLE 17.3 JavaScript Comparison Operators

You can also enter conditional expressions in the Console to see their results. As you'll see, they all return either `true` or `false`. For example, `1 == 2` will evaluate as `false`, and `5 > 3` will evaluate as `true`. If you're curious about the result of a conditional expression, you can always test it there.

You can also use the `!` operator, which reads as “not,” to negate any Boolean expression. For example, if you enter **`!true`** in the Console, the result of the expression will be `false`. If you want to negate an expression that uses a conditional operator, you'll need to use parentheses. For example, try this expression in the Console:

```
!(1 == 2)
```

The result will be `true`.

Loops

You'll occasionally want a group of statements to be executed more than once. JavaScript supports two kinds of loops. The first, the `for` loop, is ideal for situations in which you want to execute a group of statements a specific number of times. The second, the `while` loop, is useful when you want a set of statements to be executed until a condition is satisfied.

for Loops Here's a `for` loop:

[Click here to view code image](#)

```
for (var count = 1; count <= 10; count++) {
    console.log("Iteration number " + count);
}
```

The loop starts with the `for` keyword, followed by all the information needed to specify how many times the loop body will be executed. (Trips through a loop are referred to as *iterations*.) Three expressions are used to define a `for` loop. First, a variable is declared to keep track of the loop iterations. The second is a Boolean expression (evaluates to `true` or `false`) that terminates the loop when it is `false`. The third is an expression that increments the loop counter so that the loop condition will eventually be satisfied. In the preceding example, I declared the variable `count` with an initial value of 1. I specified that the loop

will execute until the value of `count` is no longer less than or equal to 10. Then I used an operator you haven't seen, `++`, to increment the value of `count` by one each time through the loop.

The body of the loop uses `console.log` to print out a message in the console every time the loop executes. The `console` object refers to the Console in the web developer tools, and the `log` method prints whatever is passed as an argument to the console directly. It's useful for debugging, especially when you add it to your scripts within pages to get a sense of the state of your scripts while they're running.

Caution

As you can see, the `for` statement is self-contained. The `count` variable is declared, tested, and incremented within that statement. You shouldn't modify the value of `count` within the body of your loop unless you're absolutely sure of what you're doing.

while Loops The basic structure of a `while` loop looks like this:

[Click here to view code image](#)

```
var color = 'blue';
while (color == 'blue') {
    console.log("Color is still blue.");
    if (Math.random() > 0.5) {
        color = 'not blue';
    }
}
```

Tip

One thing you may notice when you're entering code in the Console is that multiline statements will return an error if you enter part of the statement and press Return. To enter multiline statements yourself, you can either paste them into the Console instead of typing them, or you can press Shift-Return at the end of the lines to indicate that they are part of a multiline block. When you press the Return key at the end, all the lines will be evaluated together.

The `while` loop uses only a condition. The programmer is responsible for creating the condition that will eventually cause the loop to terminate somewhere inside the body of the loop. It might help you to think of a `while` loop as an `if` statement that's executed repeatedly until a condition is satisfied. As long as the `while` expression is true, the statements inside the braces following the `while` loop continue to run forever—or at least until you close your web browser.

In the preceding example, I declare a variable, `color`, and set its value to "blue". The `while` loop will execute until it is no longer true that `color` is set to "blue". Inside the loop, I print a message indicating that the color is still blue, and then I use an `if` statement that may set the `color` variable to a different value. The condition in the `if` statement uses `Math.random()`, another method of the `Math` object that returns a

value between 0 and 1. In this case, if it's greater than 0.5, I switch the value so that the loop terminates.

If you prefer, you can write `while` loops with the condition at the end, which ensures that they always run once. These are called `do... while` loops and look like this:

```
var color = "blue";
do {
  // some stuff
}
while (color != "blue");
```

Even though the test in the loop will not pass, it will still run once because the condition is checked after the first time the body of the loop runs.

Caution

When you're using `while` loops, avoid creating infinite loops. This means that you must manipulate one of the values in the looping condition within the body of your loop. If you do manage to create an endless loop, about the only option you have is to shut down the web browser. If you're going to iterate a specific number of times using a counter, it's usually best to just use a `for` loop.

Functions

Functions are a means of grouping code together so that it can be called whenever you like. To create a function, you declare it. The following code includes a function declaration:

[Click here to view code image](#)

```
function writeParagraph(myString) {
  document.write("<p>" + myString + "</p>");
}
```

A function declaration consists of the `function` keyword, a function name, a list of parameters the function accepts (in parentheses), and the body of the function (enclosed in curly braces). This function is named `writeParagraph`, and it accepts a single parameter, `myString`. Function parameters are variables that are accessible within the body of the function. As you can see, this function prints out the value passed in as an argument inside a `<p>` tag. After I've declared this function, I can then use the following code later in the page:

[Click here to view code image](#)

```
writeParagraph("This is my paragraph.");
```

It will produce the output:

```
<p>This is my paragraph.</p>
```

Note

When it comes to the values passed to functions, you'll see them referred to as *parameters* or as *arguments*. Technically, the variables listed in the function declaration are parameters, and the values passed to the function when it is called are arguments.

Functions can be written to accept multiple arguments. Let's look at another function:

[Click here to view code image](#)

```
function writeTag(tag, contents) {  
    document.write("<" + tag + ">" + contents + "</" + tag + ">");  
}
```

This function accepts two arguments: a tag name and the contents of that tag. There's one special statement that's specific to functions: the `return` statement. It is used to specify the return value of the function. You can use the value returned by a function in a conditional statement, assign it to a variable, or pass it to another function. Here's a function with a return value:

[Click here to view code image](#)

```
function addThese(value1, value2) {  
    return value1 + value2;  
}
```

Here are a couple of examples of how you might use that function:

[Click here to view code image](#)

```
if (addThese(1, 2) > 10) {  
    document.write("Sum is greater than 10.");  
}  
var sum = addThese(1, 2);
```

One other thing to note is that the values passed to function as arguments are copies of those values unless the arguments are objects.

Here's one more example, and the results are shown in [Figure 17.6](#):

Input ▼

[Click here to view code image](#)

```
<script language="JavaScript">  
function modifyValue(myValue) {  
    document.write(myValue + "<br>");  
    myValue = "new value";  
    document.write(myValue + "<br>");  
}  
  
var value = "old value";  
modifyValue(value);  
document.write(value + "<br>");  
</script>
```

Output ▼

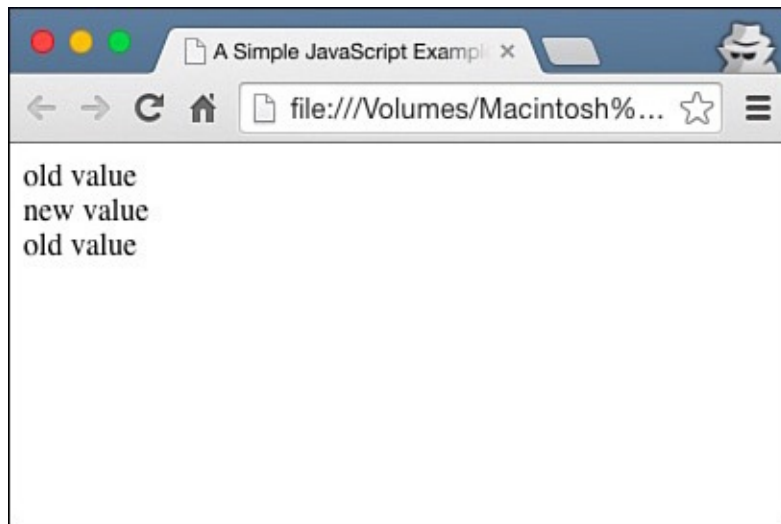


FIGURE 17.6 Values passed to functions are copies.

Functions are called using the function name, followed by parentheses. If you are passing arguments to a function, they are included in the parentheses in a comma-separated list. Even if you're not using arguments, the parentheses are still required. This is true whether you're calling a function you wrote yourself or a function that's built in to JavaScript.

Data Types

I've mentioned JavaScript's type system, but I haven't talked much about JavaScript data types. JavaScript supports the following types of values:

- Strings, like "Teach Yourself Web Publishing".
- Boolean values (true or false).
- Numbers, integer or decimal.
- `null`, which is used to represent an unknown or missing value.
- `undefined`, the value associated with variables that have been declared but have not yet had a value assigned to them. Also the return value of methods that don't return anything.

This is the full set of primitive data types that JavaScript supports. JavaScript attempts to convert data to whatever type it needs in a given context. So if you take a Boolean value and use it in a context where a string is expected, JavaScript will convert it to a string. In some cases, this automatic conversion process can lead to odd results. For example, if you try to use a value that's not a number in a context where a number is expected, JavaScript will return a special value, `NaN`, which is short for "not a number":

[Click here to view code image](#)

```
Math.sqrt("a string"); // The value of squareRoot is NaN
```

Boolean values represent a state of either true or false. You've already seen some examples that involve Boolean values. For example, `if` statements and `while` loops require conditional expressions that return a Boolean value. Any JavaScript value or expression can ultimately be converted to a Boolean. The values that are treated as false are the number zero, empty strings, `null`, `undefined`, and `NaN`. Everything else is

true.

To explicitly convert data from one type to another, you can type casting functions. They are `Number()`, `Boolean()`, and `String()`. Type casts to `Boolean` are the most interesting because they allow you to see the `Boolean` value of an expression. For example, to confirm that `NaN` is false, you could type the following in the Console:

```
Boolean(Math.sqrt("a"))
```

Arrays

Arrays are lists of things. They can be lists of values, lists of objects, or even lists of lists. There are a couple of ways to declare arrays. The first is to create your own `Array` object, like this:

```
var list = new Array(10);
```

That declares an array with 10 slots. Arrays are numbered (or indexed) starting at 0, so an array with ten elements has indexes from 0 to 9. You can refer to a specific item in an array by placing the index inside square brackets after the array name. So, to assign the first element in the array, you use the following syntax:

```
list[0] = "Element 1";
```

If you want to add elements to your array when you declare it, you can use what's called an array literal, like this:

[Click here to view code image](#)

```
var list = ["red", "green", "blue"];
```

To find out how many elements are in an array, you can use a property of the array called `length`. Here's an example:

```
listLength = list.length
```

Objects

You've already been introduced to a few objects—most recently, the `Array` object. JavaScript features a number of built-in objects, and the browser supplies even more (as discussed in the next section). The first thing you need to know about objects is that they have properties. You just saw one property: the `length` property of the `Array` object.

Object properties are accessed through what's known as dot notation. You can also access properties as though they are array indexes. For example, if you have an object named `car` with a property named `color`, you can access that property in two ways:

```
car.color = "blue";  
car["color"] = "red";
```

You can also add your own properties to an object. To add a new property to the `car` object, I just have to declare it:

```
car.numberOfDoors = 4;
```

There are a number of ways to create objects, but you should stick to the best one. To create an object, you can use an object literal, which is similar to the array literal I just

described:

[Click here to view code image](#)

```
var car = { color: "blue", numberOfDoors: 4, interior: "leather" };
```

That defines an object that has three properties. As long as the properties of the object follow the rules for variable names, there's no need to put them in quotation marks. The values require quotation marks if their data type dictates that they do. You can name properties whatever you like, though, as long as you use quotation marks.

In addition to properties, objects can have methods. Methods are just functions associated with the object in question. This may seem a bit odd, but methods are properties of an object that contain a function (as opposed to a string or a number). Here's an example:

[Click here to view code image](#)

```
car.description = function() {  
    return color + ' car ' + ' with '  
        + numberOfDoors + ' and a ' + interior + ' interior';  
}
```

As you can see, this is a bit different from the function declarations you've seen before. When you declare a method, instead of specifying the function name in the `function` statement, you assign an anonymous function to a property on your object. You can specify parameters for your methods just as you specify them for functions.

After you've added a method to an object, you can call it in the same way the methods of built-in objects are called. Here's how it works:

[Click here to view code image](#)

```
document.write(car.description());
```

Note

The core JavaScript language contains lot of built-in objects—too many to cover here. For more information about these objects, look at the JavaScript documentation provided by Mozilla or Microsoft.

The JavaScript Environment

I've taken you on a very brief tour of the JavaScript language, but beyond the basic language syntax, which involves declarations, control structures, data types, and even core objects that are part of the JavaScript language, there's also the browser environment. When your scripts run, they have access to the contents of the current page, to other pages that are open, and even to the browser itself. I've mentioned the `document` object, which provides access to the contents of the current page.

Now let's look at a specific object. The top-level object in the browser environment is called `window`. The `window` object's children provide information about the various elements of a web page. Here are some of the most commonly used children of `window`:

- location** Contains information about the location of the current web document, including the URL and components of the URL, such as the protocol, domain name, path, and port.
- history** Holds a list of all the sites that a web browser has visited during the current session and gives you access to built-in functions that enable you to send the user forward or back within the history.
- document** Contains the complete details of the current web page. All the tags and content on the page are included in a hierarchy under `document`. Not only can you examine the contents of the page by way of the `document` object, but you can also manipulate the page's contents, too.

You can find a complete list of the available objects in the Mozilla JavaScript documentation at <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.

Because the entire browser environment is accessible through this hierarchical set of objects, you can access anything as long as you know where it lives in the hierarchy. For example, all the links on the current page are stored in the property `document.links`, which contains an array. Each of the elements in the array has its own properties as well, so to get the location to which the first link in the document points, you use `document.links[0].href`.

Events

All the examples you've seen so far are executed as soon as the page loads. JavaScript is about making your pages more interactive, which means writing scripts that function based on user input and user activity. To add this interactivity, you need to bind your JavaScript code to events. The JavaScript environment monitors user activity and provides the opportunity for you to specify that code will be executed when certain events occur.

There are three ways to bind JavaScript code to an event handler. The first is to use an HTML attribute. For example, if you want to display an alert when a user clicks a link, you could bind the event like this:

[Click here to view code image](#)

```
<a href="/whatever.html" onclick="alert('Be careful')">Link</a>
```

The disadvantage of this approach is that it makes your JavaScript hard to maintain. You wind up with JavaScript scattered throughout your documents, and it makes it difficult to reuse the same JavaScript code on many pages. You should not use this approach for any but the simplest things.

The second is to bind the event to the appropriate property of the element. If you have a reference to a link called `myLink` in JavaScript, you can add a click event like this:

[Click here to view code image](#)

```
myLink.onclick = function(event) { alert('Be careful'); };
```

This approach enables you to separate your JavaScript from your HTML, but it allows you to bind only one function to the click event of the link. This can come back to bite you in the future or cause problems if you include other JavaScript code on your page that might

also need to bind an event to the click event of that link.

Finally, you can use the `addEventListener()` method to attach an event handler to an event on an element. Here's an example:

[Click here to view code image](#)

```
myButton.addEventListener('click', function(){ alert('Be careful!'); },
false);
```

You can use `addEventListener()` to attach any number of event handlers to the same event on the same element. It's the preferred method for event binding currently. If you need to support older browsers like Internet Explorer 8, there are a number of JavaScript libraries that will handle the binding of event handlers to events in a safe, cross-platform way. In the next lesson, you will learn how to use one of the most popular libraries, jQuery.

[Table 17.4](#) provides a list of the event handlers that JavaScript provides.

Event Handler	When It's Called
<code>onblur</code>	Whenever a visitor leaves a specified form field
<code>onchange</code>	Whenever a visitor changes the contents of a specified form field
<code>onclick</code>	Whenever a visitor clicks a specified element
<code>onfocus</code>	Whenever a visitor enters a specified form field
<code>onload</code>	Whenever a page and all of its images have finished loading
<code>onmouseover</code>	Whenever a visitor places the mouse cursor over a specified object
<code>onselect</code>	Whenever a visitor selects the contents of a specified field
<code>onsubmit</code>	Whenever a visitor submits a specified form
<code>onunload</code>	Whenever the current web page is changed

TABLE 17.4 JavaScript Event Handlers

First, let me explain how to bind an event using HTML attributes. All the event handlers listed in [Table 17.4](#) can be used as attributes for tags that respond to them. For example, the `onload` handler is associated with the `body` tag. As you know, JavaScript code is executed as soon as it is encountered. Suppose you want to write a script that modifies all the links on a page. If that script is executed before all the links have been loaded, it will miss some of the links. Fortunately, there's a solution to this problem. The `onload` event does not occur until the entire page has loaded, so you can put the code that modifies the links into a function and then bind it to the page's `onload` event. Here's a page that uses `onload`:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
  <title>Modifying Links with JavaScript</title>
</head>
<body onload="linkModifier()">
```



```

<ul>
  <li><a href="http://google.com/">Google</a></li>
  <li><a href="http://www.nytimes.com/">New York Times</a></li>
</ul>
<script type="text/javascript">
  function linkModifier() {
    for (var i = 0; i < document.links.length; i++) {
      document.links[i].href = "http://example.com";
    }
  }
</script>
</body>
</html>

```

This page contains a script tag, and that script tag contains a single function declaration. The function, `linkModifier()`, changes the `href` attribute of all the links on the page to `http://example.com/`. To access the links, it uses `document.links`, which is a reference to an array of all the links in the document. It iterates over each of those links, changing their `href` properties from the values specified in the HTML to the new URL. The main point of this example, though, is the `onload` attribute in the `body` tag, which contains the call to `linkModifier()`. It's important to associate that call with the `onload` event so that all the links on the page have been processed before the function is called. If I'd put the function call inside the `<script>` tag, the function call might have occurred before the page was loaded.

Most often, when using attributes to bind events, function calls are used, but the value of the attributes can be any JavaScript code, even multiple statements, separated by semicolons. Here's an example that uses the `onclick` attribute on a link:

[Click here to view code image](#)

```

<a href="http://google.com/" onclick="alert(this.href); return
false;">Google</a>

```

In this example, the value of the `onclick` attribute contains two statements. The first uses the built-in `alert()` function to display the value in the `href` attribute of the link. The second prevents the link from taking the user to a new page. So clicking the link will display the alert message in [Figure 17.7](#) and do nothing after the user acknowledges the alert.

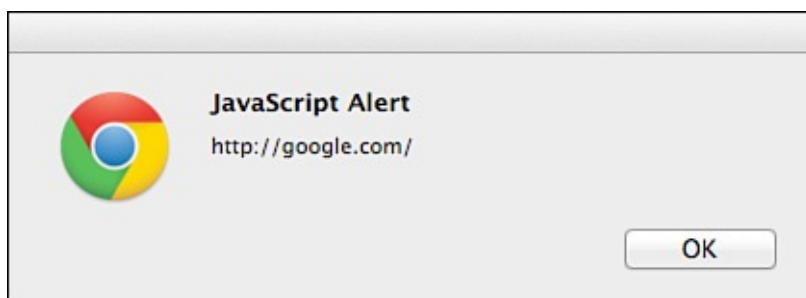


FIGURE 17.7 A JavaScript alert message.

Whether you're writing code in your event binding attribute or writing a function that will be used as an event handler, returning `false` will prevent the default browser action for that event. In this case, it prevents the browser from following the link. If the `onsubmit` action for a form returns `false`, the form will not be submitted.

The Meaning of `this`

You might be a bit puzzled by the use of `this` as a variable name in an event handler. Here, `this` is shorthand for the current object. When you're using an event handler in a tag, `this` refers to the object represented by that tag. In the previous example, it refers to the link that the user clicked on. The advantage of using `this` is that it places the event in a useful context. I could use the same attribute value with any link and the code would still work as expected. It's particularly useful when you're using functions as event handlers and you want to make them easy to reuse.

At one time, using event-handler attributes to bind functions to events was the most common approach, but these days, it's more common to bind events to elements in other ways. It's considered poor style to include JavaScript throughout your web pages, and using the event-handler attributes can override event bindings that are applied from JavaScript rather than in the HTML. In [Lesson 18](#), I explain how to bind events to elements without changing your markup.

Learning More About JavaScript

The list of statements, functions, and options included in this lesson represents only part of the potential offered by JavaScript.

For this reason, I cannot overemphasize the importance of the online documentation provided by Mozilla. All the latest JavaScript enhancements and features will be documented first at <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.

Summary

JavaScript enables HTML publishers to include simple programs or scripts within a web page without having to deal with the many difficulties associated with programming in high-level languages such as Java or C++.

In this lesson, you learned about the `<script>` tag and how it's used to embed JavaScript scripts into an HTML document. In addition, you explored the basic structure of the JavaScript language and how to use JavaScript in the browser environment.

With this basic knowledge behind you, in the next lesson, you'll explore some real-world examples of JavaScript and learn more about the concepts involved in JavaScript programming.

Workshop

The following workshop includes questions, a quiz, and exercises related to JavaScript.

Q&A

Q Don't I need a development environment to work with JavaScript?

A Nope. As with HTML, all you need is a text editor and a browser that supports JavaScript. You might be confusing JavaScript with Java, a more comprehensive programming language that needs at least a compiler for its programs to run. However, tools like the Chrome Developer Tools, FireBug for Firefox, and Safari's Web Inspector can make your life easier. Consult the documentation on those tools to learn more about their JavaScript features.

Q What is AJAX?

A One topic we haven't covered yet is AJAX. AJAX is a term used to describe scripts that communicate with the server without requiring a web page to be fully reloaded. For example, you can use it to fetch information and display it on the page or to submit a form for processing, all without changing the full page in the browser.

Q When I use JavaScript, do I need to accommodate users whose browsers may not support JavaScript or who have disabled it?

A Some estimates indicate that over 95% of web users have JavaScript enabled. However, unless you have a really good reason not to, you should make accommodations for users without JavaScript. You need not offer users who don't have JavaScript an identical experience to those who have it, but they should be able to access your site. For example, if you run an online store, do you really want to shut out users because of their browser configuration?

Q In Java and C++, I previously defined variables with statements such as `int`, `char`, and `String`. Why can't I do this in JavaScript?

A As I mentioned previously, JavaScript is a loosely typed language. This means that all variables can take any form and can even be changed on-the-fly. As a result, the context in which the variable is used determines its type.

Quiz

- 1.** What HTML tag is used to embed JavaScript scripts in a page?
- 2.** What are *events*? What can JavaScript do with them?
- 3.** Is an expression that evaluates to the value 0 true or false? How about the string "false" inside quotation marks?
- 4.** How do you make sure that a variable you create in a function is only visible locally in that function?
- 5.** How are functions different from methods?

Quiz Answers

- 1.** To accommodate the inclusion of JavaScript programs in a normal HTML document, Netscape introduced the `<script>` tag. By placing a `<script>` tag in a document, you tell the web browser to treat any lines of text inside the tag as script rather than as content for the web page.
- 2.** Events are special actions triggered by things happening in the system (windows

opening, pages being loaded, forms being submitted) or by reader input (text being entered, links being followed, check boxes being selected). Using JavaScript, you can perform different operations in response to these events.

3. The number 0 is false, and the string "false" is true. The only false values are 0, null, an empty string, undefined, NaN (not a number), and the Boolean value false itself.
4. The var statement is used to define a local variable inside a function.
5. Methods are associated with a specific object, and functions are standalone routines that operate outside the bounds of an object.

Exercises

1. If you haven't done so already, take a few minutes to explore the documentation for JavaScript at <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. See whether you can find out what enhancements were included in the latest version of JavaScript that weren't included in earlier versions.
2. Find a simple JavaScript script somewhere on the Web—either in use in a web page or in an archive of scripts. Look at the source code and see whether you can decode its logic and how it works.

Lesson 18. Using jQuery

JavaScript is a programming language that makes it possible to manipulate the contents of web pages after they have loaded. jQuery and other JavaScript libraries make it much easier to take advantage of JavaScript. jQuery enables JavaScript programmers to code without worrying about differences among browsers. It also provides a number of other features that make it easy to build powerful JavaScript features without writing very much code. In this lesson, you'll learn about the following:

- What JavaScript libraries are and why you might want to use one
- How to use the jQuery library in your pages
- How to select elements on a page with jQuery
- How to bind events using jQuery
- How to manipulate styles on a page
- How to change the content of a page
- How to fetch content from an external source using AJAX

What Are JavaScript Libraries?

In this book, I've talked about browsers and incompatibilities between them. The popular browsers differ in their support for HTML and CSS and in their support for JavaScript. Unlike CSS and HTML, though, JavaScript can actually be used to solve the problem of incompatibilities in JavaScript implementations. You can write code that detects which browser is being used or even the specific capabilities of the browser and then add logic to make sure the program works correctly for whatever environment that it's in.

For example, some browsers allow you to retrieve elements from the document by class name using the `getElementsByClassName()` method, and others do not. If your script depends on that method, it will break in some browsers. You can work around the problem by checking to see whether the method exists before you use it, and if it doesn't, using another technique that works in the browsers that don't support it.

JavaScript libraries were created by people who had to do this sort of thing too many times and decided to package up all of these kinds of workarounds to create a simpler interface to common functionality that hides all the incompatibilities of the various browsers. In doing so, the authors also added many other features to make life more convenient for JavaScript developers. The most popular JavaScript libraries make it easier to bind code to events, select elements on the page to act on in your programs, and even make calls to the server from JavaScript to dynamically change elements on the page, using a technique referred to as AJAX.

You might have noticed that I am referring to JavaScript libraries as a group. That's because there are a number of libraries that provide roughly the same set of features. They were all independently developed and work differently from one another, and each has its own set of advantages and disadvantages. If you're starting from scratch, choosing

between them is a matter of taste.

Getting Started with jQuery

Entire books are published about each of the popular JavaScript libraries, so it would be foolish to try to cover them all in this lesson. Instead, I'm going to focus on introducing jQuery. I've chosen it mainly because it's the easiest library to get started with, especially if you already know CSS. Even if you don't wind up using jQuery, you'll still get an idea of how JavaScript libraries work by reading this section. You'll just have to follow up by digging into the documentation to learn how to apply the same concepts with the library that you use instead.

jQuery is a regular JavaScript file that you can include in your page using the `<script>` tag. To get started, download your own copy at <http://jquery.com/>. After you've downloaded jQuery, you can start using it in your pages. The easiest way to include it in a page, especially for local testing, is to rename the downloaded file to `jquery.js` and put it in the same directory as the HTML page:

[Click here to view code image](#)

```
<script type="text/javascript" src="jquery.js"></script>
```

Note

The file you download will have a different name than `jquery.js` because it will include the jQuery version number. You'll have to rename it as `jquery.js` or use the full filename in your `<script>` tag. You can download jQuery in production or development configurations. The production configuration is "minified"—compressed so that it downloads as quickly as possible. Unfortunately, the minified file is unreadable by humans; so if you think you may need to look at the jQuery source, download the development version. Just be sure to replace it with the minified version when you make your site live.

Your First jQuery Script

jQuery is built around the idea of selecting objects on the page and then performing actions on them. In some ways, it's very similar to CSS. You use a selector to define an element or set of elements, and then you write some code that is applied to those elements. Here's an example of a page that includes a simple jQuery script:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Example</title>
</head>
<body>
  <a href="1.html">A link</a>
  <script src="jquery.js"></script>
  <script>
    $(document).ready(function() {
      $("a").click(function(event) {
```

```

        alert("You clicked on a link to " + this.href );
    });
});
</script>
</body>
</html>

```

The first `<script>` tag loads the external jQuery script. The second contains the script I wrote. This script causes an alert to be displayed whenever a user clicks a link. I'll break it down line by line.

The first line is important because you'll see it or a variation of it in nearly every jQuery script:

```
$(document).ready(function() {
```

First, `$` is the name of a function declared by jQuery, and `document` is an argument to that function. The `$` function selects the set of elements matched by the selector provided as an argument. In this case, I've passed `document` as the argument, and it matches the `document` object—the root object of the page's document object model. Usually, the selector is a CSS selector, but the `document` object is an alternative that you can use, as well.

To the right, you'll see a call to the `ready` method, which is applied to the elements that the selector matches. In this case, it's the `document` object. jQuery provides convenient methods for binding events to objects or elements, and in this case, it will be used to bind an anonymous function to the document's `ready` event. The "ready" event is provided by jQuery.

The `window` object supports the `onload` event, which is what's normally used to execute JavaScript when the page is displayed. The `window.onload` event doesn't "fire" (call any methods that are bound to it) until the page has fully loaded. This can be a problem for pages that contain large images, for example. The JavaScript code won't run until the images load, and that could lead to strange behavior for users.

jQuery's `document.ready` event, however, fires when the page has been fully constructed. This can cause the JavaScript to run a bit earlier in the process, while images are being downloaded. With jQuery it's customary to perform all the work you want to occur when the page loads within an anonymous function bound to the `document.ready` event. It's so common, in fact, that a shortcut is provided to make doing so even easier. The line above can be rewritten as follows:

```
$(function() {
```

jQuery knows that you intend to bind the function to the `document.ready` event. Here's the code that's bound to the event:

[Click here to view code image](#)

```

$("a").click(function(event) {
    alert("You clicked on a link to " + this.href );
});

```

This code binds a function that prints an alert message containing the URL of the link that

the user clicked on to every link on the page. In this case, I use "a" as the selector I pass to jQuery, and it works exactly as it does with CSS. The `click()` method binds a function to the `onclick` event for all the elements matched by the selector it's called on, in this case, all the `<a>` tags on the page.

Selecting Elements from the Document

In the previous example, I used a jQuery selector to bind an event handler to all the links on a page. The important thing to know about jQuery is that jQuery selectors are a superset of CSS selectors; pretty much any selector you can use with CSS can also be used with jQuery. This is a fantastic shortcut, but it's worth discussing what happens behind the scenes a bit as well.

When a web page is loaded in the browser, the browser creates a programmatic representation of the page structure called the Document Object Model. This representation of the page is what's shown in the Elements tab of the Developer Tools. You'll notice that implied tags (like `tbody` for tables) are included and that broken HTML is repaired in this representation.

In JavaScript, the built-in `document` object is the entry point for manipulating the DOM. It provides methods like `getElementById`, `getElementByName`, and `getElementsByTagName` to enable you to query the DOM directly from JavaScript. There are also methods that enable developers to add and remove elements from the DOM and to modify them. However, using these methods is much more cumbersome than using jQuery. For example, here's some JavaScript code that accomplishes the same thing as the jQuery code above accessing the DOM directly:

[Click here to view code image](#)

```
<script>
window.addEventListener('load', function() {
    var links = document.getElementsByTagName("a");

    for (var i = 0; i < links.length; i++) {
        links[i].addEventListener('click', function() {
            alert("You clicked on a link to " + this.href );
        });
    }
});
</script>
```

As you can see, that's an awful lot more code than I wrote for the jQuery example. The other problem is that this code does not take browser incompatibilities into account. It works in most browsers, but not all of them. Most of the incompatibilities between browsers are in their DOM implementations. So if you write code to manipulate the DOM from scratch, you have to account for these differences. That would entail writing even more code.

jQuery translates the selectors that developers use into the calls it needs to make on the `document` object to retrieve the proper list of elements. Thus, you can use a selector like `a` to find all the links on a page or `a.archives` to find all the links with the class `archives`, and jQuery will access the DOM in whatever way works with the user's

browser and return the list of elements they selected.

Binding Events

Most JavaScript development involves binding event handlers to events. Whether it's making sure that JavaScript code that initializes things loads at the right time, or adding interactivity by associating code with actions the user takes on the page, you're dealing with event binding.

jQuery properly binds events so that multiple handlers can be bound to the same event, across all browsers. If you bind your events using jQuery, you never have to worry about overriding events bound in another script. Also, as you saw in the previous example, jQuery makes it easy to bind handlers to multiple elements at once. For example, the following code would disable all the links on the page:

[Click here to view code image](#)

```
$( "a" ).click(function(event) { event.preventDefault(); })
```

jQuery enables you to refer to the object representing the event itself in your handler. To do so, add the `event` parameter to the anonymous function representing the event handler. You can name the parameter whatever you like, in this case, I named it `event`. I then call the `preventDefault` method on the event. That method, provided by jQuery, indicates that the default action associated with the event should not be performed. The default action of a link is to change the browser's location to the URL in the link, but this event handler prevents that from happening.

Here's a more useful example. Let's say that I want links to external sites to open in a new window, and all the links to external sites use a fully qualified URL whereas local links do not. I could use the following event handler:

[Click here to view code image](#)

```
$(function () {
    $( "a" ).click(function (event) {
        if (null != this.href && this.href.length > 4
            && this.href.substring(0, 4) == "http") {
            event.preventDefault();
            window.open(this.href);
        }
    });
});
```

The `if` statement tests whether the `href` attribute of the link starts with "http." The other tests in the `if` statement are checks that prevent the `substring` method from raising an error if the URL is not present or is too short. If the URL does start with "http," I open a new window for that URL and prevent the default action. If it doesn't, the default action is allowed to continue. Instead of adding special classes to external links on the page or using the `onclick` attribute for each of them to open new windows, I just used jQuery's selector functionality and a bit of programming to take care of it for me.

Caution

If you can't get the previous script to work correctly, make sure that the `if` statement is all on one line: `if (null != this.href && this.href.length > 4 && this.href.substring(0, 4) == "http") {`. Some browsers don't like whitespace in the middle of `if` statements.

jQuery provides methods for binding most events to jQuery objects. For more a full list of events jQuery supports, see <http://api.jquery.com/bind/>.

Modifying Styles on the Page

Another powerful feature of jQuery is that it enables you to modify styles on the page on-the-fly. jQuery enables you to modify the page styles indirectly through convenience methods that hide and show elements, for example, and enables you to change styles directly.

Hiding and Showing Elements

For example, you can hide and show elements easily based on activity by the user. Here's a sample page that swaps out two elements whenever they are clicked:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
  <title>Anchors</title>
</head>
<body>

  <div id="open" style="padding: 1em; border: 3px solid black;
    font-size: 300%;">We are open</div>

  <div id="closed" style="padding: 1em; border: 3px solid black;
    font-size: 300%;">We are closed</div>

  <script src="jquery.js"></script>
  <script type="text/javascript" charset="utf-8">
$(function () {
  $("#closed").hide();
  $("#open, #closed").click(function (event) {
    $("#open, #closed").toggle();
  });
});
</script>
</body>
</html>
```

The page contains two `<div>`s, one containing the text "We are closed" and one containing the text "We are open." In the event handler for the document's ready state, I hide the `<div>` with the ID `closed`:

```
$("#closed").hide();
```

That method sets the `display` style of the elements matched by the selector to `none` so that when the page finishes loading, that `<div>` will not be visible, as shown in [Figure 18.1](#).



FIGURE 18.1 The initial state of the page. “We are closed” is hidden.

Then I bind an event handler to the `onclick` event of those `<div>`s containing the following code:

```
$("#open, #closed").toggle();
```

As you can see, this selector matches both the IDs `open` and `closed` and calls the `toggle()` method on each of them. That method, provided by jQuery, displays hidden items and hides items that are being displayed. So, clicking the `<div>` will cause the other `<div>` to appear and hide the one you clicked. After you click the `<div>` and the two elements have been toggled, the page appears, as shown in [Figure 18.2](#).



FIGURE 18.2 The state of the page after the element has been clicked.

Retrieving and Changing Style Sheet Properties

You can also modify styles on the page directly. If I change the event handler in the previous example to contain the following code, the text will be underlined when the user clicks the `<div>`, as shown in [Figure 18.3](#):

[Click here to view code image](#)

```
$(this).css("text-decoration", "underline");
```

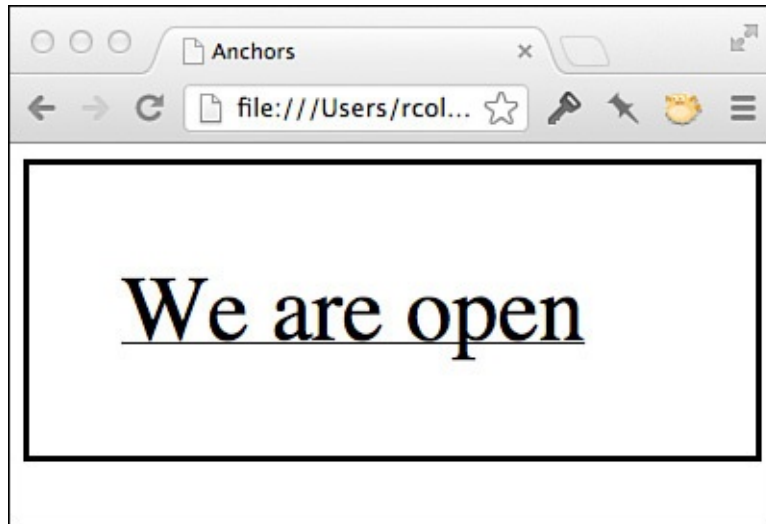


FIGURE 18.3 The text is underlined after the user clicks on the `<div>`.

jQuery enables you to manipulate any styles on the page in this fashion. You can also retrieve the values of CSS properties using the `css()` method; just don't leave out the argument. If I instead change the body of the event handler to the following, the browser will display the current font size used in the `<div>` that the user clicked:

[Click here to view code image](#)

```
alert("The font size is " + $(this).css("font-size"));
```

A browser window with the alert displayed appears in [Figure 18.4](#).

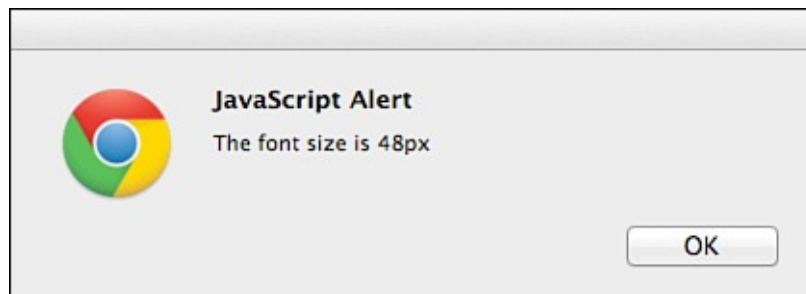


FIGURE 18.4 An alert box displaying the value of a CSS property.

Using these techniques, you can build pages with expanding and collapsing lists, add borders to links when users mouse over them, or allow users to change the color scheme of the page on-the-fly.

Modifying Content on the Page

Not only can you modify the styles on the page using jQuery, but you can also modify the content of the page itself. It provides methods that enable you to remove content from the page, add new content, and modify existing elements.

Manipulating Classes

jQuery provides a number of methods for manipulating the classes associated with elements. If your page already has a style sheet, you might want to add or remove classes from elements on-the-fly to change their appearance. In the following example page, shown in [Figure 18.5](#), the class highlighted is added to paragraphs when the mouse is moved over them, and it's removed when the mouse moves out:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
  <title>Altering Classes on the Fly</title>
  <style>
    p { padding: .5em;}
    p.highlighted { background: #666666; }
  </style>
</head>
<body>

  <p>This is the first paragraph on the page.</p>
  <p>This is the second paragraph on the page.</p>

  <script src="jquery.js"></script>
  <script type="text/javascript">
    $(function () {
      $("p").mouseenter(function () {
        $(this).addClass("highlighted");
      });

      $("p").mouseleave(function () {
        $(this).removeClass("highlighted");
      });
    })
  </script>
</body>
</html>
```

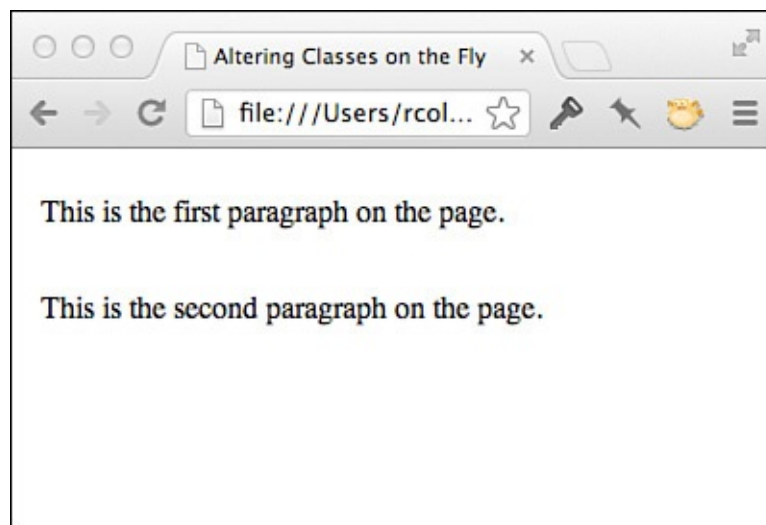


FIGURE 18.5 No paragraphs are highlighted initially.

On this page, I have two paragraphs that have no classes assigned to them by default. I also have a style sheet that applies a gray background to any paragraph with the class

highlighted. Most important, I have the following two event handlers:

[Click here to view code image](#)

```
$( "p" ).mouseenter(function () {  
    $(this).addClass("highlighted");  
});  
$( "p" ).mouseleave(function () {  
    $(this).removeClass("highlighted");  
});
```

In this example, I use the jQuery `mouseenter` and `mouseleave` events to fire events whenever the user moves his mouse over or away from a paragraph. As you can see in [Figure 18.6](#), when the user's mouse is over the paragraph, the class `highlighted` is applied to it. When the mouse moves away, the class is removed.

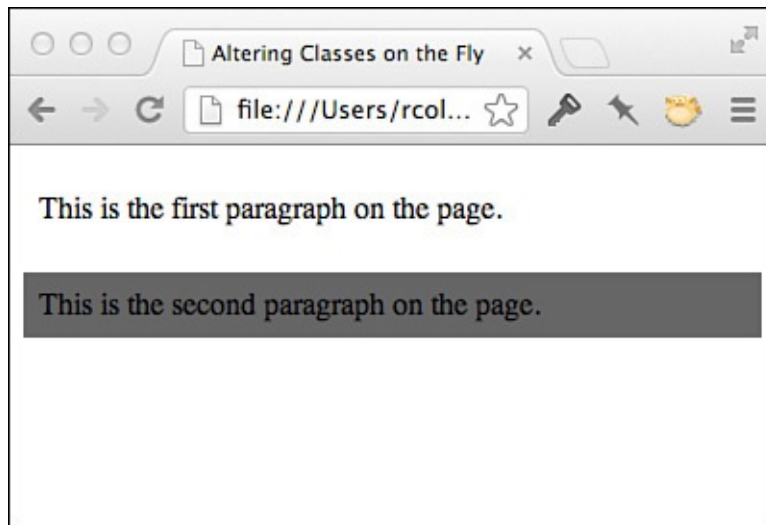


FIGURE 18.6 Paragraphs are highlighted when the mouse is over them.

You can use jQuery's `toggleClass()` method to reverse the state of a particular class on an element. In the following example, the elements in the list are highlighted the first time the user clicks them, and the highlighting is removed the next time the user clicks them. All that's required is to toggle the `highlighted` class with each click:

[Click here to view code image](#)

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Altering Classes on the Fly</title>  
    <style>  
        li.highlighted { background: yellow; }  
    </style>  
</head>  
<body>  
    <ul>  
        <li>One</li>  
        <li>Two</li>  
        <li>Three</li>  
        <li>Four</li>  
    </ul>  
  
    <script src="jquery.js"></script>  
    <script type="text/javascript" charset="utf-8">  
        $(function () {
```

```

        $("li").click(function () {
            $(this).toggleClass("highlighted");
        });
    });
</script>
</body>
</html>

```

Finally, jQuery can check for the presence of a class using the `hasClass()` method. If I change the body of the event handler in the previous example to the following function, the first time the user clicks a list item, the `highlighted` class will be applied. The second time, an alert (shown in [Figure 18.7](#)) will be displayed indicating that the item is already highlighted:

[Click here to view code image](#)

```

$("li").click(function () {
    if (!$(this).hasClass("highlighted")) {
        $(this).addClass("highlighted");
    }
    else {
        alert("This list item is already highlighted.");
    }
});

```

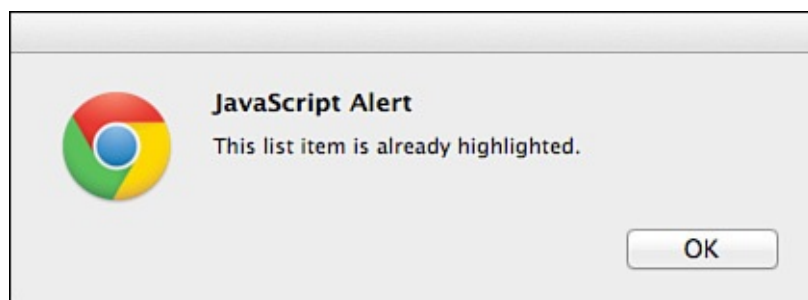


FIGURE 18.7 An alert is displayed when users click a paragraph the second time.

In this example, I use the `hasClass()` method to determine whether the class is already present. If it isn't, I add it. If it is, I display the alert.

Manipulating Form Values

You can also use jQuery to modify the contents of form fields. The `val()` method can be used to both retrieve the value of form fields and modify them. In many cases, websites put an example of the input that should be entered into a form field in the field until the user enters data. In the following example, the form starts with sample data in the field, but it's removed automatically when the user focuses on the field. If the user doesn't enter data, the sample data is restored. [Figure 18.8](#) shows the initial state of the page.

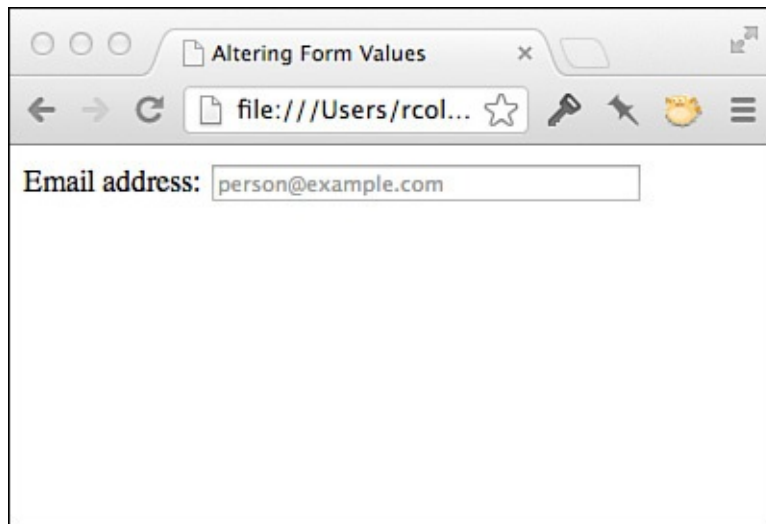


FIGURE 18.8 When the page loads, the sample content appears in the form field.

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
  <title>Altering Form Values</title>
  <style>
    input[name="email"] { color: #999; }
  </style>
</head>
<body>
  <form>
    <label>Email address: <input name="email" value="person@example.com"
size="40" /></label>
  </form>

  <script src="jquery.js"></script>
  <script type="text/javascript" charset="utf-8">
$(function () {
  $("input[name='email']").focus(function () {
    if ($(this).val() == "person@example.com") {
      $(this).val("");
      $(this).css("color", "black");
    }
  });

  $("input[name='email']").blur(function () {
    if ($(this).val() == "") {
      $(this).val("person@example.com");
      $(this).css("color", "#999");
    }
  });
});
</script>
</body>
</html>
```

Again, I use two event handlers in this example. The event handlers are new, as is the selector. Here's one of them:

[Click here to view code image](#)

```
$( "input[name='email']" ).focus(function () {
```

In this case, I'm using a selector that's based on an attribute value. It matches an input field in which the `name` attribute is set to `email`, and it binds to the `focus` event. This event fires when the user places the cursor in that field. The event handler for the focus event does two things: sets the value of the field to an empty string, and changes the color from gray to black, but only if the value is `person@example.com`. If it's something else, it's a value the user entered and should be left alone. [Figure 18.9](#) shows what the form looks like when the user initially clicks in the field.

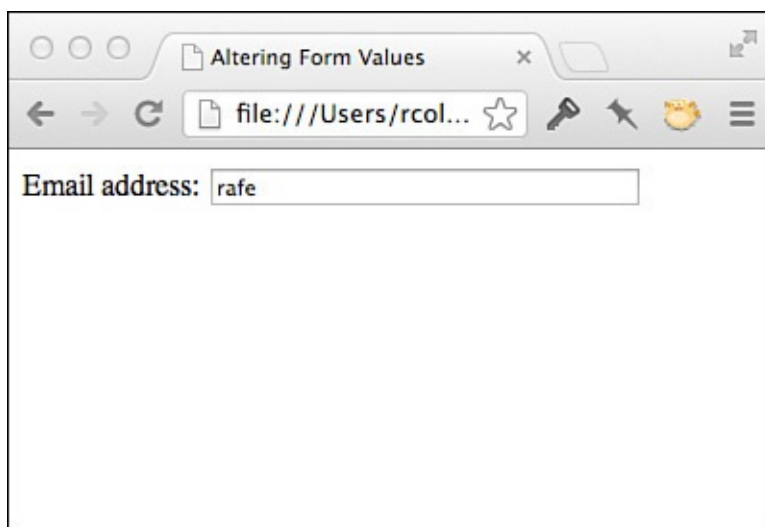


FIGURE 18.9 The contents of the email field are removed when the user clicks in it.

The other event handler is bound to the `blur` event, which fires when the cursor leaves the field. If the field has no value, it changes the color back to gray and puts the example input back into the field.

Manipulating Attributes Directly

You can also use jQuery to manipulate the attributes of elements directly. For example, disabling a form field entirely requires you to modify the `disabled` attribute of that field.

I've added a Submit button to the form from the previous example and set it to `disabled`. Here's the new form:

[Click here to view code image](#)

```
<form>
  <label>Email address: <input name="email" value="person@example.com"
size="40">
  <input id="emailFormSubmit" type="submit" disabled>
</form>
```

[Figure 18.10](#) shows the form with the sample content and the disabled Submit button.

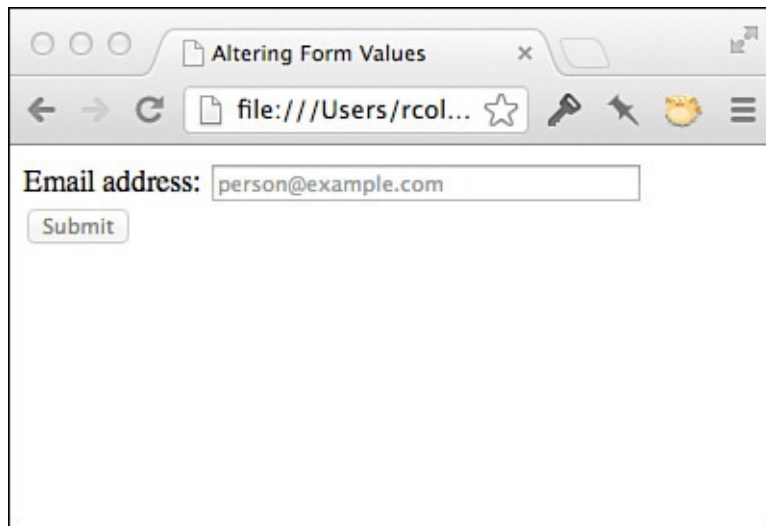


FIGURE 18.10 This form contains sample content, and the Submit button is disabled.

I only want to let users click the Submit button if they've already entered an email address. To add that check, I need to add a bit of code to the `blur` event for the email field, as shown:

[Click here to view code image](#)

```
$("input[name='email']").blur(function () {  
    if ($(this).val() == "") {  
        $(this).val("person@example.com");  
        $(this).css("color", "#999");  
        $("#emailFormSubmit").attr("disabled", "disabled");  
    }  
    else {  
        $("#emailFormSubmit").removeAttr("disabled");  
    }  
});
```

If the user leaves the field having set a value, the `disabled` attribute is removed from the Submit button, as shown in [Figure 18.11](#). If the user leaves the field without having entered anything, the `disabled` attribute is added, just in case it was previously removed.

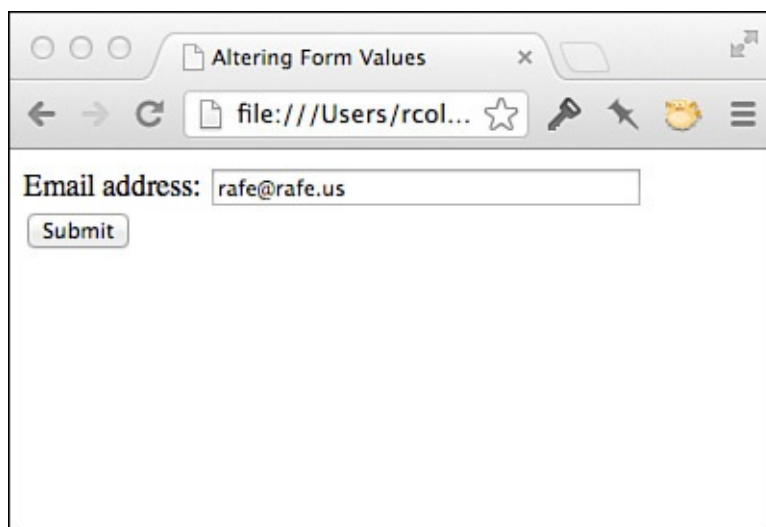


FIGURE 18.11 The Submit button is no longer disabled after an email address is entered.

Adding and Removing Content

jQuery provides a number of methods that can be used to manipulate the content on the page directly. Here's a more complex example that demonstrates several ways of manipulating the content on a page—users can add new content to the page, remove content from the page, and even wipe out all the content inside an element in one click. The initial page appears in [Figure 18.12](#).

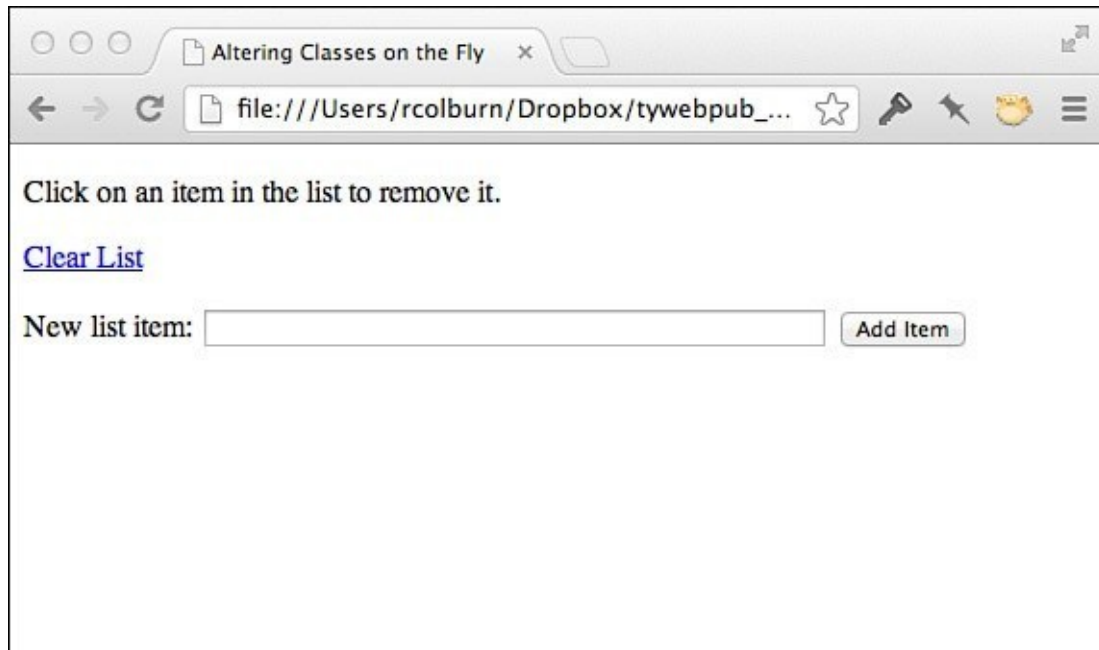


FIGURE 18.12 A page that allows you to add and remove content on the fly.

I'll start with the markup for the page. First, I need a list. In this example, the user will be able to add elements to the list and remove elements from the list. All I need is an empty list with an ID:

```
<ul id="editable">
</ul>
```

Next, I have a form that enables users to add a new item to the end of the list. It has a text input field and a Submit button:

[Click here to view code image](#)

```
<form id="addElement">
  <label>New list item: <input name="liContent" size="60" /></label>
  <input type="submit" value="Add Item" />
</form>
```

Finally, I've added a link that removes all the elements the user has added to the list:

[Click here to view code image](#)

```
<p><a id="clearList" href="#">Clear List</a></p>
```

The action is on the JavaScript side. Let's look at each of the event handlers for the page one at a time. First, here's the event handler for the Clear List link:

[Click here to view code image](#)

```
$("#clearList").click(function (event) {
  event.preventDefault();
  $("#editable").empty();
});
```

This event handler prevents the default action of the link (which would normally return the user to the top of the page) and calls the `empty()` method on the list, identified by selecting its ID. The `empty()` method removes the contents of an element.

Next is the event handler for the form, which enables users to add new items to the list:

[Click here to view code image](#)

```
$("#addElement").submit(function (event) {
    event.preventDefault();
    $("#editable").append("<li>"
        + $("#addElement input[name='liContent']").val() + "</li>");
    $("#addElement input[name='liContent']").val("");
});
```

I bind this handler to the `submit` event for the form, just as I did in the previous example. First, it prevents the form submission from completing its default action—submitting the form to the server. Then I append the content to the list. I select the list using its ID, and then I call the `append()` method, which adds the content in the argument just inside the closing tag for the elements that match the selector. In this case, I put the value in the text field, which I obtain using the `val()` method that you’ve already seen, inside an opening and closing `<li>` tag, and pass that to the `append()` method. I also remove the content from the text field because it has been appended to the list. [Figure 18.13](#) shows the list once a few elements have been added.

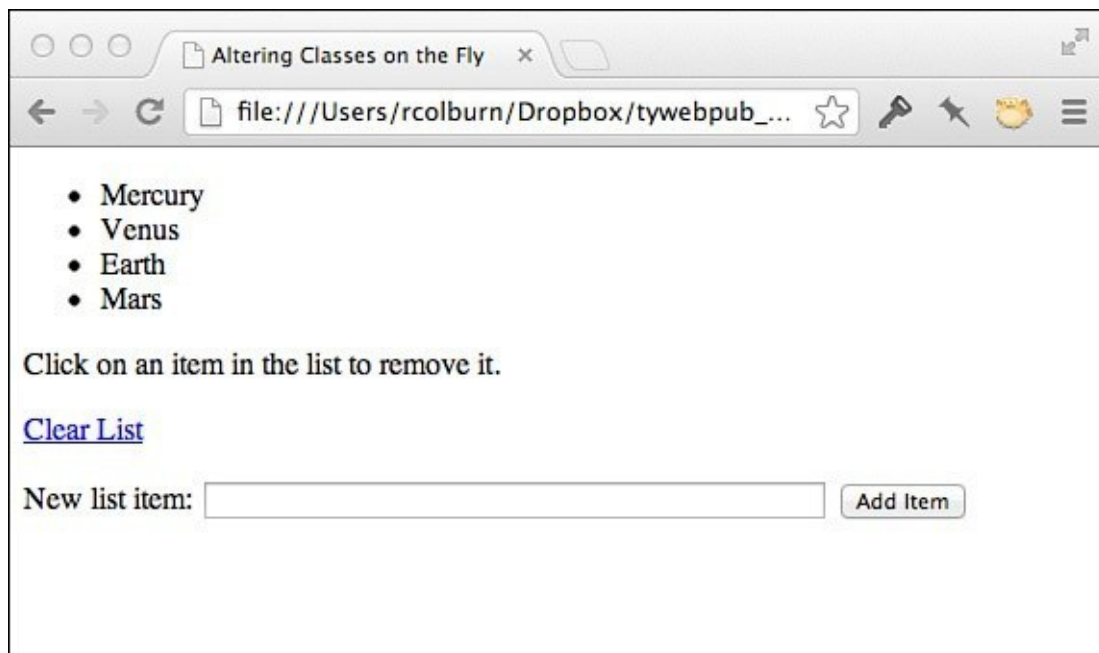


FIGURE 18.13 A page with items added by a user.

Finally, I allow users to remove items from the list by clicking them. There’s one trick here. As you’ve seen, to do so I’ll need to use the `click` handler for the `<li>` elements in the list. In this case, there’s a catch. When the page loads and the `document.ready` event for the page initially fires, there are no elements in the list to bind events to. Fortunately, jQuery provides a way to set up an event handler so that it’s automatically bound to newly created elements on the page. Here’s the code:

[Click here to view code image](#)

```
$(document).on('click', "#editable li", function () {
    $(this).remove();
```



```
});
```

As you can see, the event binding is slightly different here. Instead of using the `click()` method, I've used the `on()` method with `document` as the selector. This indicates that I want to monitor changes to the page and perform the event binding that follows any time an element matching the selector (the second argument) is added. The first argument is the name of the event to bind—it's the name of the event to be bound, placed in quotation marks. The second argument is the selector to match. The third is the event handler as it would normally be written. The `on()` method is one of the most powerful features of jQuery because it enables you to automatically treat dynamically generated content the same way you'd treat content that's on the page at the time that it loads.

Here's the full source for the page:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
  <title>Altering Form Values</title>
</head>
<body>
  <ul id="editable">
  </ul>
  <form id="addElement">
    <label>New list item: <input name="liContent" size="60" /></label>
    <input type="submit" value="Add Item" />
  </form>
  <p><a id="clearList" href="#">Clear List</a></p>

  <script src="jquery.js"></script>
  <script type="text/javascript" charset="utf-8">
$(function () {
  $("#clearList").click(function (event) {
    event.preventDefault();
    $("#editable").empty();
  });
  $("#addElement").submit(function (event) {
    event.preventDefault();
    $("#editable").append("<li>"
      + $("#addElement input[name='liContent']").val() + "</li>");
    $("#addElement input[name='liContent']").val("");
  });
  $(document).on('click', "#editable li", function () {
    $(this).remove();
  });
});
</script>
</body>
</html>
```

There are other methods for adding content in different locations in relation to a selected element. For example, if I change the `append()` call to `prepend()`, new items will be added to the top of the list rather than the bottom. You can also use the `before()` method to add content before another element, and the `after()` element to add it after. The difference is that when you use those methods, the content is placed outside the tags matched by the selector, rather than inside those tags.

Special Effects

It can be a little jarring when elements just appear or disappear instantly. Most JavaScript libraries, including jQuery, provide a library of effects that enable you to animate transitions on the page when items appear, disappear, or move. jQuery has a few basic effects built in to the core library. Supplemental effects are also available as part of jQuery UI, which you can obtain at <http://jqueryui.com/>.

The four effects that are part of jQuery are fade in, fade out, slide up, and slide down. I'm going to build on the previous example to show you how they can be used to soften the transitions when you add items to the page or remove items from it. Adding the effects to the page just requires a few small tweaks to the event handlers that I already created.

The first effect I added applies the fade-out effect when users click a list item to remove it. To cause an element to fade out, you call the `fadeOut()` method on the results of a selector that matches that element. Here's the code:

[Click here to view code image](#)

```
$(document).on('click', "#editable li", function () {  
    $(this).fadeOut('slow', function() { $(this).remove() });  
});
```

When you call `fadeOut()`, it sets the `display` property for the element to `none`—essentially, it's a fancy replacement for `hide()`. [Figure 18.14](#) shows a list item that's in the process of fading out.

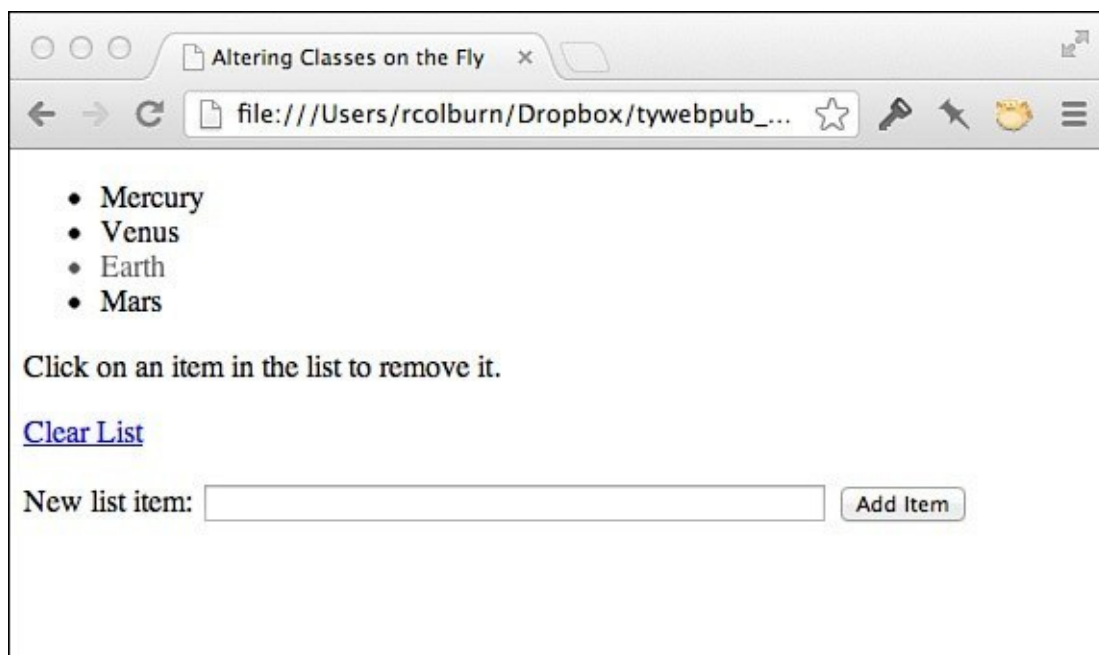


FIGURE 18.14 The jQuery fade-out effect in progress.

In this case, I want to actually remove the element from the page entirely. To do so, I need a callback, which is included as the second argument to `fadeOut()`. The callback is run whenever the animation is complete, and in this case, it removes the element from the page. The first argument is used to specify the speed of the animation. Setting it to `slow` means that it will take 600 milliseconds to complete. By default, the animation takes 400 milliseconds. You can also set it to `fast` (200 milliseconds), or you can enter a number of milliseconds yourself.

I've also updated the event handler for the Clear List link. In this case, I use the slide-up effect, shown in [Figure 18.15](#), when the list is cleared. Here's the updated event handler:

[Click here to view code image](#)

```
$("#clearList").click(function (event) {  
    event.preventDefault();  
    $("#editable").slideUp('slow', function () {  
        $("#editable").empty()  
        $("#editable").show();  
    });  
});
```

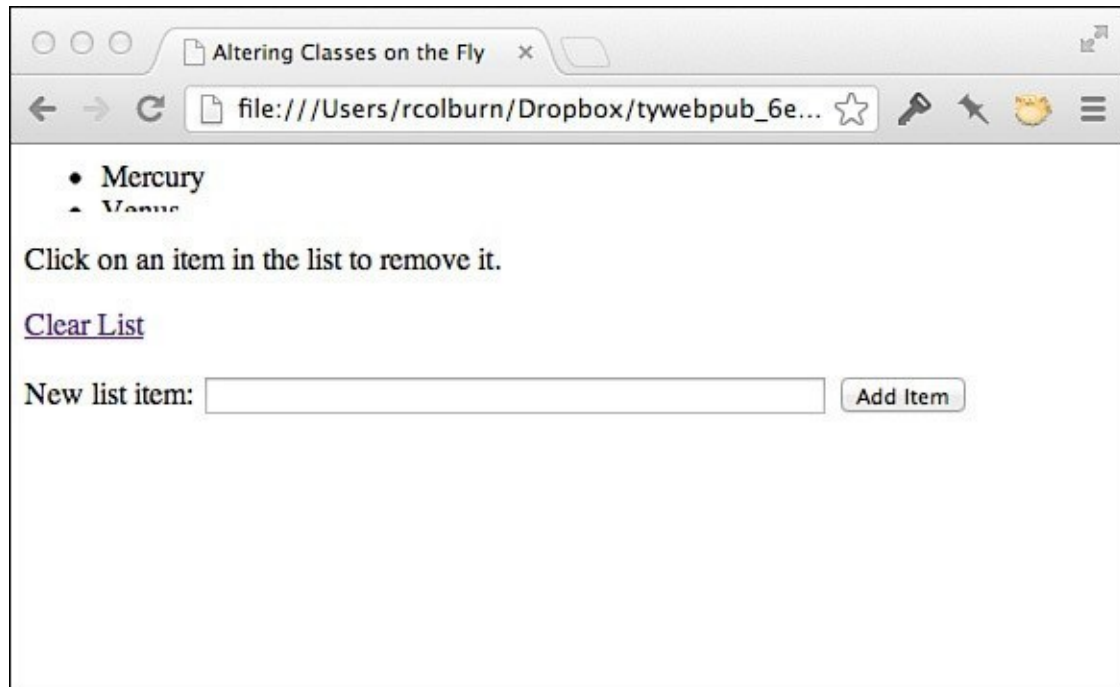


FIGURE 18.15 The jQuery slide-up effect.

The changes here are similar to those for the previous event handler. After the animation is complete and the list is hidden, I call the `empty()` method to remove the contents of the list and then call `show()` on the now hidden list so that when the user adds new elements to it, the list will be visible.

Finally, I want the new items I add to the list to fade in rather than just appearing. Here's the updated event handler with the `fadeIn()` call included:

[Click here to view code image](#)

```
$("#addElement").submit(function (event) {  
    event.preventDefault();  
    var content = "<li>" + $("#addElement input[name='liContent']").val() +  
    "</li>";  
    $(content).hide().appendTo("#editable").fadeIn('slow').css("display",  
    "list-item");  
    $("#addElement input[name='liContent']").val("").focus();  
});
```

This event handler is a little bit more complex. First, I initialize a new variable with the content to add to the page, just to make the code a little more readable. Then I go through all the steps required to fade the new content in. At this point, I should explain one of the other nifty features of jQuery—method chaining. Nearly all jQuery methods return the object of the method. So if I use `hide()` to hide something, the method returns whatever

it was that I hid. This makes it convenient to call multiple methods on the same object in succession.

In this case, I call `hide()`, `appendTo()`, `fadeIn()`, and `css()` on the jQuery object representing the new content that I created. First, I pass the content variable to `$()`, which allows me to call jQuery's methods on the content. Then I call `hide()` on it so that it doesn't appear instantly when I append it to the list.

After that, I use `appendTo()` to append it to the list. The difference between `append()` and `appendTo()` is that with `append()`, the object of the method is the selector that represents the container, and the method parameter is the content to be appended, whereas with `appendTo()`, the content to be appended is the object and the selector for the container is the method parameter. In this case, using `appendTo()` makes it easier to chain all of these method calls.

After I've appended the hidden content to the list, I call `fadeIn('slow')` to make it gradually appear. Then, finally, I call `css("display", "list-item")` on the new content. When `fadeIn()` is done, it sets the `display` property for the list item to `block`, which causes the bullet for the list item not to appear in some browsers. Setting the `display` property to `list-item` ensures that a bullet is displayed.

AJAX and jQuery

One of the primary reasons programmers started adopting JavaScript libraries was that they made it much easier to use AJAX techniques on their websites and applications. What's AJAX? It's a description for functionality that uses a JavaScript feature called `XmlHttpRequest` to make requests to the server in the background and use the results within the page.

The Web is based around the concept of pages. When you click a link or submit a form, usually you leave the page that you're on and go to a new page with a different URL (or refresh the current page). AJAX is about retrieving content from the server and then placing it on the page using JavaScript.

The previous example demonstrated how to use JavaScript to process data entered in a form and add it to a page instantly. Using AJAX, you can use the same techniques to retrieve data from the server and add it to the page. It's possible to write the code necessary to do this sort of thing from scratch, but jQuery makes it a whole lot easier.

Usually, AJAX is associated with server-side applications. For example, you can create a search engine and then use AJAX to retrieve search results and present them without ever leaving the current page. Unfortunately, there's not enough space in this book to teach you how to create a search engine. jQuery also provides the ability to retrieve information from a different static page using AJAX and present it in the current page. I'm going to present an example that takes advantage of that feature to show you how AJAX can be used.

Using AJAX to Load External Data

I've created a simple page, shown in [Figure 18.16](#), that allows users to look up information about South American countries. When a user clicks one of the links, the information about that country is retrieved from the server and displayed inline on the page.

Note

Because of the way AJAX works, this example will only work if it's deployed on a web server. If you load the files directly in your browser, the JavaScript code that retrieves the information won't work. This is because browsers have a security feature that prevents JavaScript on one server from loading content from another server.

The sample consists of two files. The first is the page shown in [Figure 18.16](#), which loads the data from the second page.

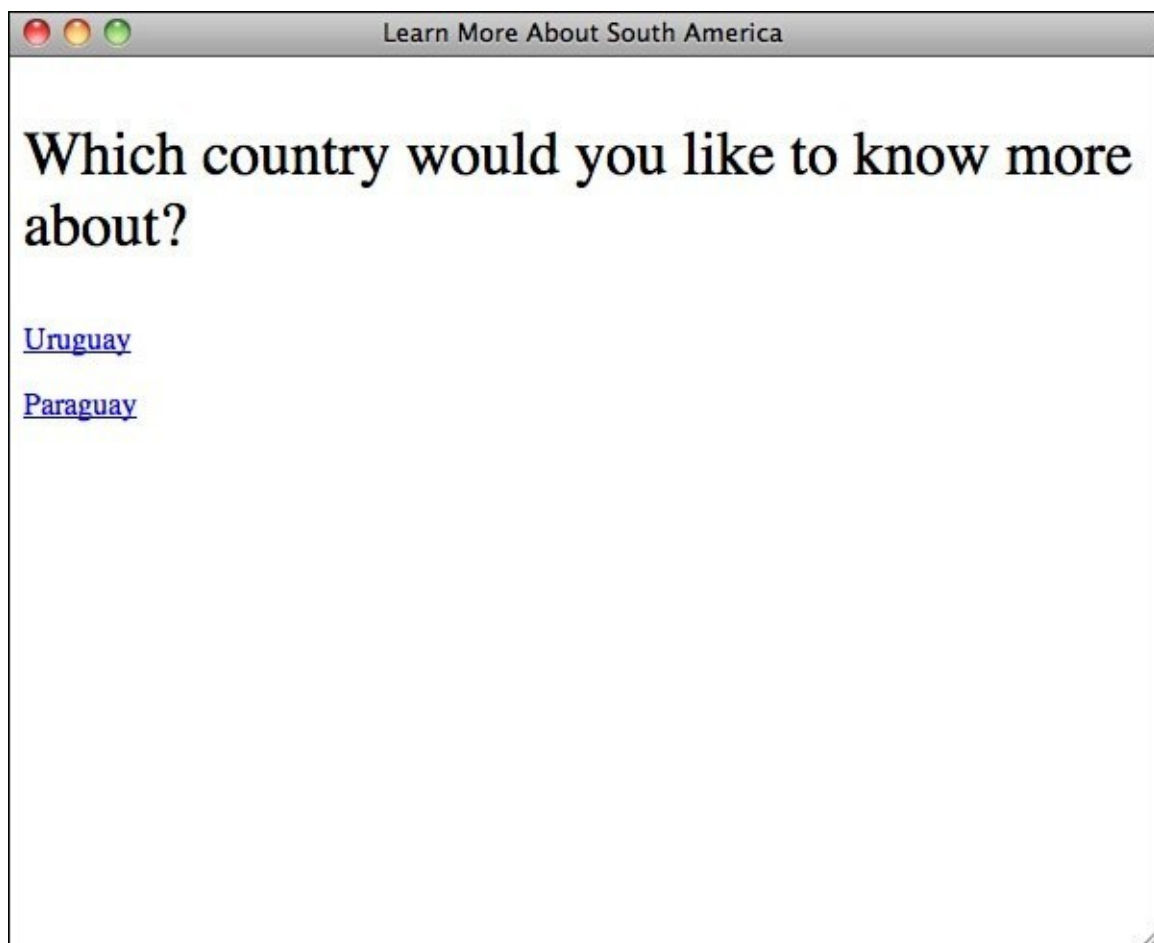


FIGURE 18.16 A page that loads data from an external source using AJAX.

The second is the page containing the information about the countries. Here's the source for the second page, `countries.html`, which contains information about the countries:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
  <title>South American Countries</title>
```

```

</head>
<body>
<div id="uruguay">
  <h2>Uruguay</h2>

  <p>Uruguay, officially the Oriental Republic of Uruguay, is a country
  located
  in the southeastern part of South America. It is home to some 3.5 million
  people,
  of whom 1.4 million live in the capital Montevideo and its metropolitan area.
  An
  estimated 88% of the population are of European descent.</p>
  <p>Uruguay's only land border is with Rio Grande do Sul, Brazil, to the
  north.
  To the west lies the Uruguay River, to the southwest lies the estuary of Rio
  de
  la Plata, with Argentina only a short commute across the banks of either of
  these
  bodies of water, while to the southeast lies the southern part of the
  Atlantic
  Ocean. Uruguay, with an area of approximately 176 thousand km2, is the second
  smallest nation of South America in area after Suriname.</p>
</div>

<div id="paraguay">
  <h2>Paraguay</h2>
  <p>Paraguay, officially the Republic of Paraguay, is a landlocked country
  in
  South America. It is bordered by Argentina to the south and southwest, Brazil
  to
  the east and northeast, and Bolivia to the northwest. Paraguay lies on both
  banks
  of the Paraguay River, which runs through the center of the country from
  north to
  south. Because of its central location in South America, is sometimes
  referred to
  as Corazon de America, or the Heart of America.</p>
  <p>As of 2009 the population was estimated at 6.3 million. The capital and
  largest city is Asuncion. The official languages are Spanish and Guarani,
  both
  being widely spoken in the country. Most of the population are mestizos.</p>
</div>
</body>
</html>

```

For a real application, instead of this simple page, you'd have a more robust service that could return lots of information about every country in South America on demand. This sample page illustrates the concept without requiring knowledge of server-side programming.

Now that the raw information is in place to be used on the page, I'll explain how the page works. When a link is clicked, the information is retrieved from `countries.html` and displayed on the initial page, as shown in [Figure 18.17](#).

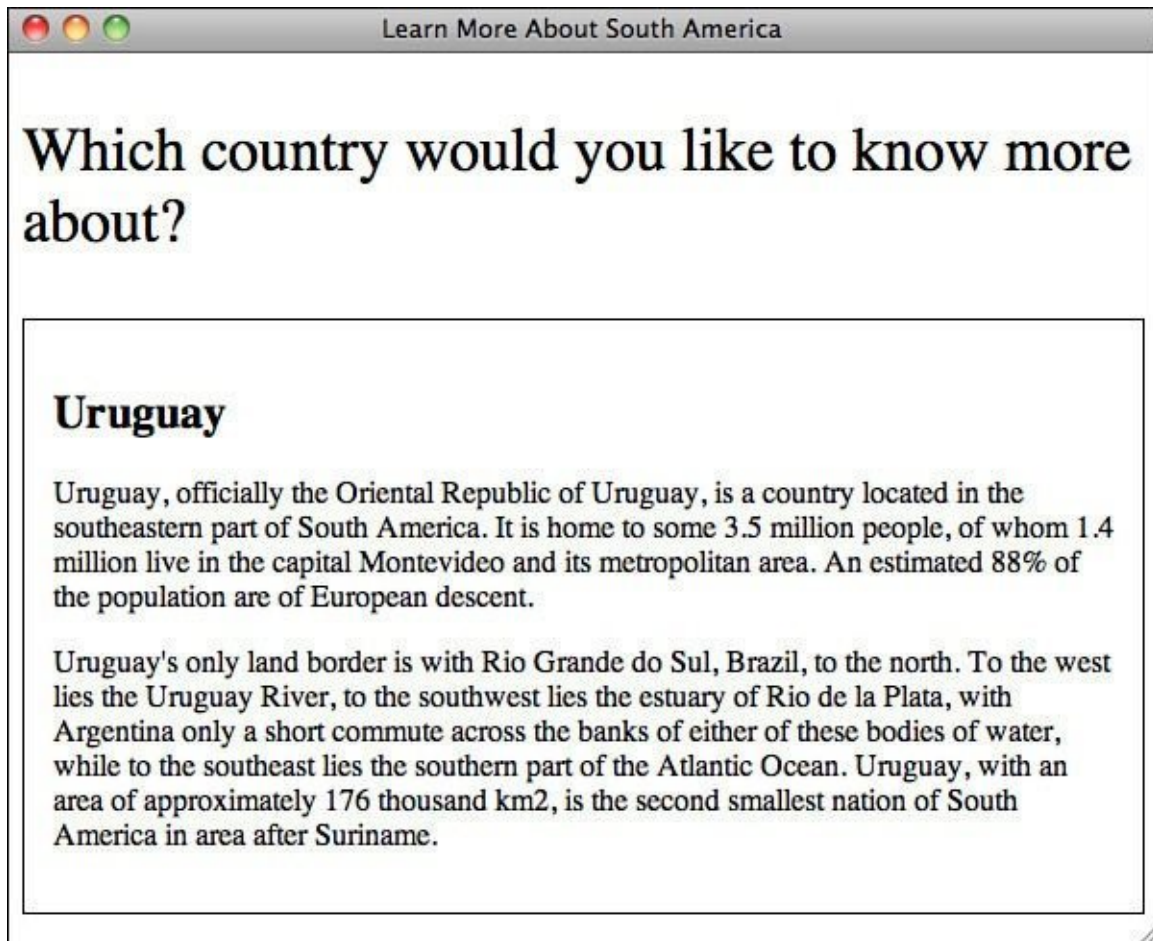


FIGURE 18.17 The information about Uruguay was loaded from an external source.

First, let's look at the two links:

[Click here to view code image](#)

```
<p class="countryOption"><a href="countries.html #uruguay">Uruguay</a></p>
<p class="countryOption"><a href="countries.html #paraguay">Paraguay</a></p>
```

They almost look like regular links. The one difference is that I've included a space between the filename and the anchor in the URL. That's because it's not really an anchor; it's the ID of a `<div>` on the `countries.html` page.

Here's the event handler for the click event for the links:

[Click here to view code image](#)

```
$( "p.countryOption a" ).click( function ( event ) {
    event.preventDefault();
    $( "p.countryOption" ).fadeOut();
    $( "#country" ).load( $( this ).attr( 'href' ) );
    $( "#country" ).fadeIn();
});
```

You should be used to most of this by now. The first line prevents the link from actually taking you to the link referenced in the `href` attribute. The second line fades out the links, because they'll be replaced by the country data.

The third line actually performs the AJAX request. It instructs jQuery to load whatever is in the `href` of the link the user clicked on into the element with the ID "country." In this case, the links refer to jQuery selectors of sorts. Remember the URLs in the links? They consist of two parts, the first being the file to load, and the second being a jQuery selector,

in this case, the ID of the country that I'll be displaying information about. jQuery loads the entire page, and then applies the selector to it to extract the information I care about.

Here's the full source code for the page:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
  <head>
    <title>Learn More About South America</title>
    <style>
      #country { border: 1px solid black; padding: 15px; }
      p.question { font-size: 200%; }
    </style>
  </head>
  <body>
    <p class="question">
      Which country would you like to know more about?
    </p>

    <div id="country">Foo</div>

    <p class="countryOption"><a href="countries.html #uruguay">Uruguay</a></p>
    <p class="countryOption"><a href="countries.html #paraguay">Paraguay</a></p>

    <script src="jquery.js"></script>
    <script type="text/javascript" charset="utf-8">
      $(function () {
        $("#country").hide();

        $("p.countryOption a").click(function (event) {
          event.preventDefault();
          $("p.countryOption").fadeOut();
          $("#country").load($(this).attr('href'));
          $("#country").fadeIn();
        });
      });
    </script>
  </body>
</html>
```

To read about other AJAX-related methods offered by jQuery, take a look at the jQuery API documentation. Most of the other jQuery methods are more suitable to application development, but they essentially work in a similar fashion to the `load()` method that you saw here.

Summary

In this lesson, I explored some of the powerful features common to most JavaScript libraries using jQuery. You learned which JavaScript libraries are available and why you might want to use them. You also learned how to include jQuery in a web page and take advantage of its functionality through the `document.ready()` event. I explained how event binding works with jQuery and how to dynamically modify the styles on a page as well as the content of a page itself. Finally, I explained what AJAX is and how jQuery and other JavaScript libraries enable you to make requests to external data sources from within a web page.

Workshop

As always, we wrap up the lesson with a few questions, quizzes, and exercises. Here are some questions and exercises that should refresh what you've learned about jQuery.

Q&A

Q Won't adding jQuery cause my pages to load more slowly?

A Yes, adding any JavaScript library will add to your overall page size. However, the browser will cache the external JavaScript file, so users should only have to download it once, when they get to the first page of your site. When they go to subsequent pages, the JavaScript library will already be in the cache.

Another option to help speed up your pages that use jQuery is to use a content delivery network or CDN. These are repositories of many different JavaScript libraries that you can link to in your web pages to take advantage of the global cache. Chances are your customers have already visited a page that uses a CDN for jQuery so it's already in their cache and you can take advantage of that. The CDN I use for jQuery is at <https://code.jquery.com/>. Simply link to the version of jQuery you want to use on your site the way you would any other script file. For example, to link to the 2.1.4 version you would write: `<script src="https://code.jquery.com/jquery-2.1.4.min.js"></script>`.

Q What about users who don't have JavaScript enabled?

A It's generally agreed that less than 3% of users have JavaScript disabled these days. However, that is still a lot of people, so you'll want to make sure that essential functionality still works for users who don't have JavaScript access. That's one of the big advantages of the unobtrusive JavaScript approach that jQuery reinforces. The markup should work fine without the JavaScript, which enhances the experience but is not essential to making the pages work.

Quiz

1. What is the purpose of jQuery's `document.ready()` handler?
2. How do I select an item with the ID `navigationLink` in jQuery?
3. What does the variable `this` refer to in an event-handling function?

Quiz Answers

1. The `document.ready()` event, provided by jQuery, fires when the HTML document is fully constructed and before resources like images are loaded. It's generally a safe point at which you can begin initializing JavaScript for a page.
2. jQuery uses CSS-style selectors. So to select an item with the ID `navigationLink`, you'd use `$("#navigationLink")`.
3. In an event handler, `this` refers to the object that the event was fired from. So if

it's a click event on a link, `this` refers to the link that was clicked.

Exercises

1. Download jQuery and use the `<script>` tag to load it in a web page.
2. Use jQuery to disable all the links on a web page.
3. Use jQuery to cause a border to appear around all the links on a web page when the user mouses over them. Make sure to remove the borders when the user moves the pointer away from the link.
4. Try to add a link to a web page that uses AJAX to retrieve the local temperature from a weather site for your city. You'll need to find the URL for a page with your city's weather on it and then create the correct selector to extract only the information you want from that page. After you find the weather page, view the source on it to figure out how to extract the information using a jQuery selector.

Lesson 19. Using JavaScript in Your Pages

Now that you have some understanding of what JavaScript is all about, you're ready to look at some practical applications of JavaScript.

In this lesson, you'll learn how to complete the following tasks:

- Validate the contents of a form
- Create a list that expands and collapses
- Add new content to a web page

Validating Forms with JavaScript

Remember the sample form that you created back in [Lesson 12](#), “[Designing Forms](#)”? It's shown again in [Figure 19.1](#). It's a typical registration form for a website, with several required fields.

Registration Form

Please fill out the form below to register for our site. Fields with bold labels are required.

Name

Gender ☐ male ☐ female

Operating System

Toys ☐ Digital Camera ☐ MP3 Player ☐ Wireless LAN

Portrait No file chosen

Mini Biography

FIGURE 19.1 The registration form.

What happens when this form is submitted? In the real world, a script on the server side validates the data that the visitor entered, stores it in a database, and then thanks the visitor

for her time.

But what happens if a visitor doesn't fill out the form correctly—for example, she doesn't enter her name or choose a value for gender? The script can check all that information and return an error. The good thing about server-side validation is that it's reliable. The web developer can be completely certain that the user's input has been validated before the script tries to store it in the database. However, validating the form input in the browser before it's submitted has some advantages as well. The validation can be designed to occur when it makes the most sense for the form in question, as opposed to waiting for the user to click the Submit button. Furthermore, saving round trips to the server conserves resources and offers better performance for users.

▼ Exercise 19.1: Form Validation

Now take a look at how the registration form is validated with JavaScript. Whenever you click the Submit button on a form, two events are triggered: the `onclick` event for the button and the `onsubmit` event for the form. For form validation, `onsubmit` is the better choice because some forms can be submitted without clicking the Submit button. When the `onsubmit` event is fired, the validation function is called.

First open the HTML for your form. It looks like this:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
<title>Registration Form</title>
<style type="text/css" media="screen">
    form div {
        margin-bottom: 1em;
    }

    div.submit input {
        margin-left: 165px;
    }

    label.field {
        display: block;
        float: left;
        margin-right: 15px;
        width: 150px;
        text-align: right;
    }

    input[type="text"], select, textarea {
        width: 300px;
        font: 18px Verdana;
        border: solid 2px #666;
        background-color: #ada;
    }

    div.required label.field {
        font-weight: bold;
    }
</style>
</head>
<body>
```

```

        div.required input, div.required select {
            background-color: #6a6;
            border: solid 2px #000;
            font-weight: bold;
        }
    </style>
</head>
<body>
<h1>Registration Form</h1>

<p>Please fill out the form below to register for our site. Fields
with bold labels are required.</p>

<form action="/register" method="post" enctype="multipart/form-data"
id="registrationForm">

    <div class="required">
        <label class="field" for="name">Name</label>
        <input name="name" type="text" />
    </div>

    <div class="required">
        <label class="field">Gender</label>
        <label><input type="radio" name="gender" value="male" /> male</label>
        <label><input type="radio" name="gender" value="female" />
female</label>
    </div>

    <div class="required">
        <label class="field">Operating System</label>
        <select name="os">
            <option value="windows">Windows</option>
            <option value="macos">Mac OS</option>
            <option value="linux">Linux</option>
            <option value="other">Other ...</option>
        </select>
    </div>

    <div>
        <label class="field">Toys</label>
        <label><input type="checkbox" name="toy" value="digicam" /> Digital
Camera
</label>
        <label><input type="checkbox" name="toy" value="mp3" /> MP3
Player</label>
        <label><input type="checkbox" name="toy" value="wlan" /> Wireless
LAN</label>
    </div>

    <div>
        <label class="field">Portrait</label>
        <input type="file" name="portrait" />
    </div>

    <div>
        <label class="field">Mini Biography</label>
        <textarea name="bio" rows="6" cols="40"></textarea>
    </div>

    <div class="submit">

```

```
<input type="submit" value="register" />
</div>
</form>
</body>
</html>
```

I'll be using jQuery to write the form validation code. So, I'll add the following code to the page right above the closing `</body>` tag:

[Click here to view code image](#)

```
<script src="https://code.jquery.com/jquery-2.1.4.min.js"></script>
```

This loads jQuery from a CDN so that your page will load faster.

Then load the `form.js` script. I placed mine in the `scripts` subdirectory, like this:

[Click here to view code image](#)

```
<script src="scripts/form.js"></script>
```

In this script I use jQuery's `document.ready` handler to bind an anonymous function to the submit event of the form, which has the ID `registrationForm`:

[Click here to view code image](#)

```
$(function() {
    $("#registrationForm").submit(function (event) {
        alert("Form submitted");
        event.preventDefault();
    });
});
```

The Validation Function When you write an event-handling function for jQuery, you can specify that the function accepts an argument, which represents the event that the handler is handling. In this case, I named the argument `event` for simplicity. In the handler, I display an alert, just to demonstrate that the JavaScript is working, and I call the `event.preventDefault()` method. This is a special method provided by jQuery that indicates that the default behavior associated with the event should not occur. In the case of a form submission, it prevents the form from being submitted. When the form validation is complete, the default action will only be prevented when the form input is not valid.

Testing For Required Fields The form has three required fields: Name, Gender, and Operating System. The validation function is responsible for checking that they each contain a value. The `name` field is a text input field. Here's the markup for the field:

[Click here to view code image](#)

```
<input name="name" type="text" />
```

Before writing the test to verify that the form field is not empty, the trick is accessing the value in the field. I've updated my event handler to look like this:

[Click here to view code image](#)

```
$("#registrationForm").submit(function (event) {
    event.preventDefault();
```



```
    console.log($(this.name).val());  
});
```

I'm still preventing the form's submission, and now I'm using the `console.log()` method to log the value of `$(this.name).val()`. The `console.log()` method is provided as a convenience so that you can print out your own messages to the Console tab in the Development Tools window. In this case, I want to print out the value of the form field. Remember that `this` is an object representing the form. The form object has properties representing each of its fields, so I can access a field with the name "name" using `this.name`. jQuery provides the `val()` method to normalize accessing the values of form fields. So, `$(this.name).val()` returns the current value of the field named "name" in the form. That line prints it on the console.

Using this information and an `if` statement, you can test the contents of `name` to see whether a name has been entered:

[Click here to view code image](#)

```
if (!$(this.name).val()) {  
    event.preventDefault();  
    alert("You must enter a name.");  
}
```

The expression in the `if` statement evaluates the value of the form field in a Boolean context. So if the field's value is null or an empty string, the expression will be false. In that case, the function displays an error message and prevents the form from being submitted.

If the name is valid, the next step is to validate the gender radio button group. Validation of these fields differs slightly because radio button groups consist of multiple fields with the same name. So in this case, I need to alter my selector to access the value of the radio button field. Remember that only one element in a set of radio buttons can be selected. I can use the CSS selector for attribute values to locate the selected radio button. The code to validate the radio button looks like this:

[Click here to view code image](#)

```
if (!$(this).find("input[name='gender']:checked").length) {  
    event.preventDefault();  
    alert("You must select your gender.");  
}
```

I access the Gender field on the form the same way I accessed the Name field. Then I used the jQuery `find` method to search for elements with the "checked" attribute checked using the `:checked` selector. I could also pass the context for a jQuery query as a second argument to the jQuery method, like this:

[Click here to view code image](#)

```
$("input[name='gender']:checked", this)
```

If neither radio button is selected, the length of the matched set will be zero and therefore be false in the context of the `if` statement, and the error message will be

displayed.

The form indicates that the Operating System field is also required, but because it's impossible to not select an item in that list, there's no need to validate it.

You can use the JavaScript debugger built in to Chrome (or whichever browser you're using) to inspect this function further. In Chrome, open the Developer Tools and click on the Sources button. Then click the button on the upper left, just below the toolbar, to view a list of the scripts on the page. From there, select `form.js` from the list. You should see a view like the one in [Figure 19.2](#).

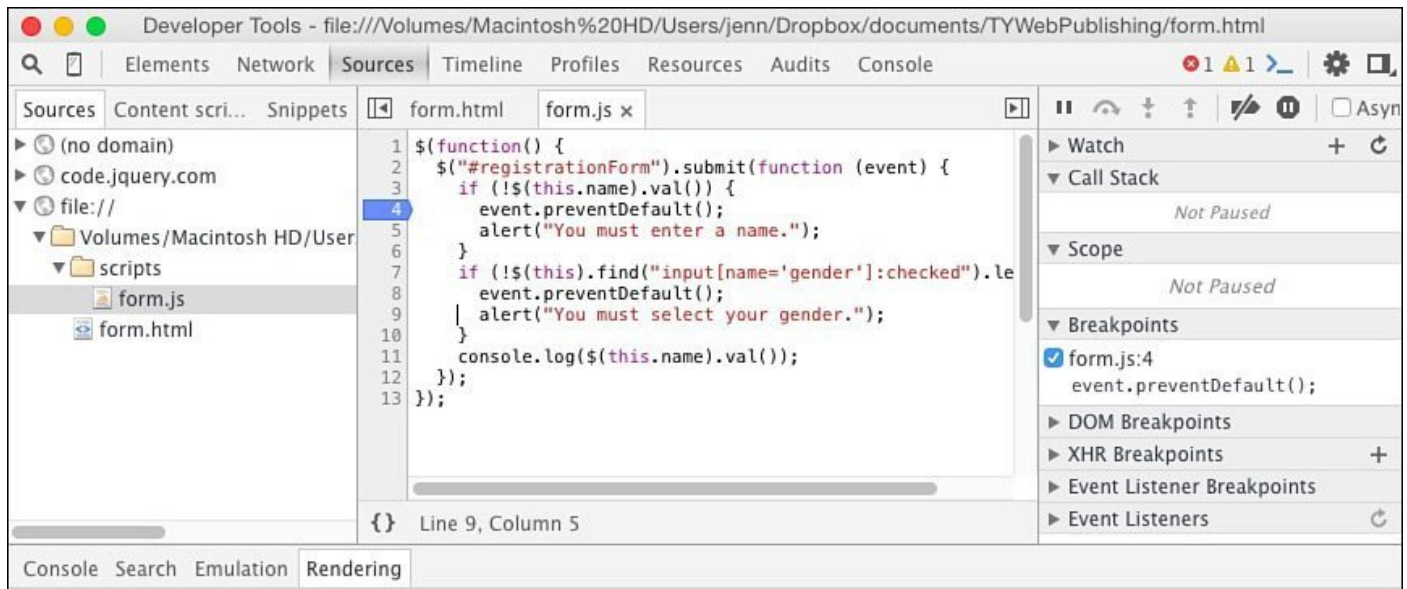


FIGURE 19.2 The Chrome Developer Tool Sources view.

In the middle pane, you can see the source code for the script. Each line of the script is numbered. If you click the line number for line 3, an arrow will appear. This sets what's called a *breakpoint*. Before the browser executes this line of the script, it will stop execution and let you inspect the inner workings of the script as it runs. This is incredibly useful when you're looking into problems in your scripts. You can see which variables are set and what their values are, and you can control execution of the script from that point on, resuming execution or proceeding one line at a time. After you've set the breakpoint on line 3, click the Submit button on the form. The Developer Tools window will be updated to look as it does in [Figure 19.3](#).

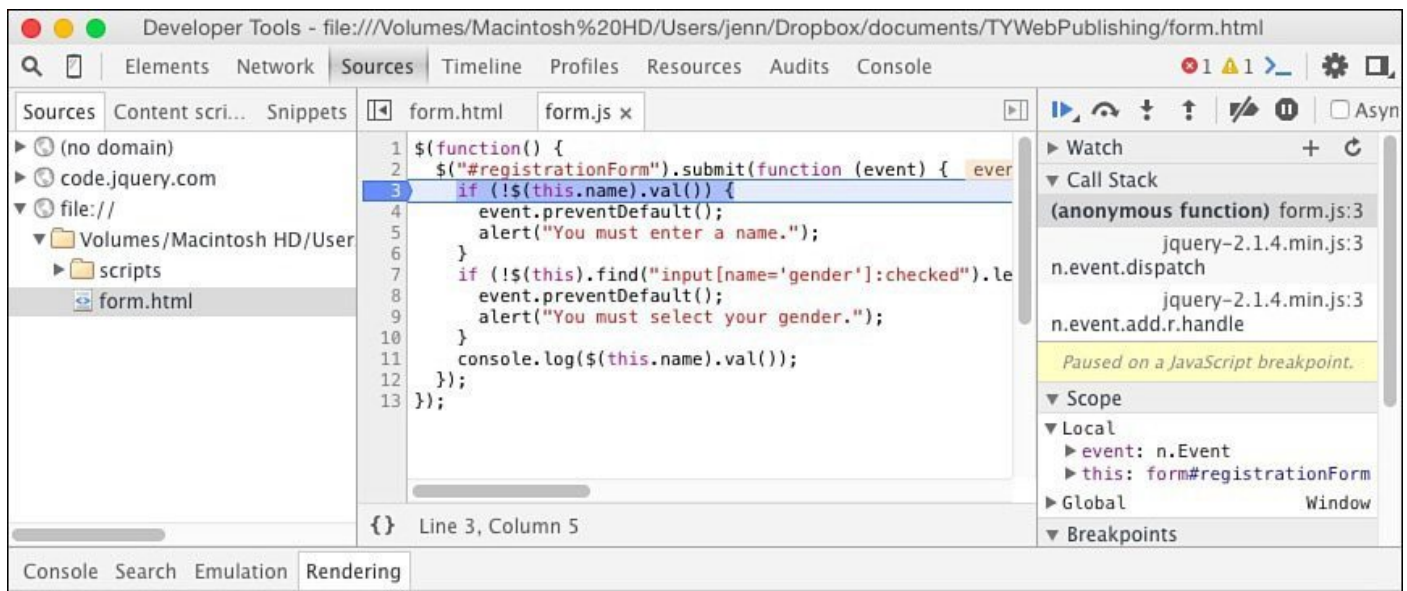


FIGURE 19.3 Running a script in the debugger.

The blue arrow next to the line number for line 3 indicates that it is the where the debugger is stopped. The buttons above the right pane allow you to control execution from there. The buttons, from left to right, are Resume, Step Over, Step Into, Step Out, and Deactivate Breakpoints. The Deactivate button causes subsequent breakpoints to be ignored. The Resume button continues execution until the script finishes or it hits another breakpoint. Step Over moves to the next statement in the current function, and Step Into debugs whatever function is being called from the current line (if there is a function call on that line). Step Out executes the remainder of the current function and resumes debugging in the calling function. You should experiment with all of these to get a feel for how the debugger works.

You'll find three pieces of information in the right pane useful immediately. The first is the *Call Stack*. It shows which function calls led to the current statement being called. When you click Step Out, the debugger moves up to the next statement in the call stack. The next is *Scope Variables*, which enables you to inspect any variables that are currently in scope. In this case, there's `event`, which is the parameter of your event handler, and `this`, which is set to the element that the event handler is bound to. If you inspect it, you'll see that in this context, `this` is the `form` that's been submitted. You'll need to access it to validate the user input in the form. Finally, the *Breakpoints* section shows all of the breakpoints that have been set. Sometimes it's hard to remember which breakpoints you've set and where you set them. You can use the Breakpoints list to unset them. You should definitely experiment with the debugger. Being able to use it effectively makes it very easy to track down many kinds of bugs.

The Completed Registration Form with JavaScript Validation When the JavaScript script that you just created is integrated with the original registration form document from [Lesson 12](#), the result is a web form that tests its contents before they're transmitted to the server for further processing. This way, no data is sent to the server until everything is correct. If a problem occurs, the browser informs the user (see [Figure 19.4](#)).



FIGURE 19.4 An alert message.

There was no need to change the source code for the page at all other than to add the JavaScript tags to load the jQuery library and the custom script for the page, as mentioned previously.

Then there's the custom script itself. Here are the full contents:

[Click here to view code image](#)

```
$(function() {  
    $("#registrationForm").submit(function (event) {  
        if (!$("this.name").val()) {  
            event.preventDefault();  
            alert("You must enter a name.");  
        }  
        if (!$("this").find("input[name='gender']:checked").length) {  
            event.preventDefault();  
            alert("You must select your gender.");  
        }  
        console.log($(".name").val());  
    });  
});
```

Improving Form Validation The form validation code in this exercise works, but it could be better. The biggest problem is that it shows an alert for each field with an error. A better approach is to show an error on the page and highlight the fields that need to be corrected before the page can be submitted. The validation code is the same; what's different is what happens when errors are found.

In this case, I'm going to print an error message above the form, and I'm going to mark each field with invalid input to indicate that it needs to be corrected. Before I get to the JavaScript, I'm going to add a few styles to the style sheet that will be used for the validation messages. Here are the new styles:

```
div.error label {  
    color: red;  
}  
  
div.errors {  
    border: 2px solid red;  
    color: red;  
    width: 50%;  
    padding: 10px;  
}
```

The first style is applied to labels inside `div` elements with the `error` class. The second style is applied to a `div` with the class `errors`. This should provide some

insight into how the validation code will work. When a field is found to have invalid input, an error message will be saved so that it can be placed in the `errors` `div`, and the class `error` will be applied to the `div` containing that field.

The conditional code in the validation JavaScript is the same. The first change I'm making is to get rid of the lines that display the alert box and replace them with the lines that add the appropriate class to the document. The updated JavaScript looks like this:

[Click here to view code image](#)

```
$(function() {
  $("#registrationForm").submit(function (event) {
    if (!$(this.name).val()) {
      $(this.name).parents("div.required").addClass("error");
      event.preventDefault();
    }

    if (!$("input[name='gender']:checked", this).length) {
      $("input[name='gender']",
this).parents("div.required").addClass("error");
      event.preventDefault();
    }
  });
});
```

The trick here is to write the proper selectors to add the error class to the appropriate element. First, here's the line that adds the error class to the enclosing `div` for the Name field:

[Click here to view code image](#)

```
$(this.name).parents("div.required").addClass("error");
```

I start by selecting the Name field. Then I use jQuery's `parents()` method to apply the `div.required` selector to all the ancestors of the form field. It matches the appropriate element, and then I use the `addClass()` method to apply the `error` class to the `div`. If you try this out, you'll see that when you submit invalid data the labels for the invalid fields suddenly turn red.

Now let's look at printing a proper error message on the page. First, I declare a variable in which to collect the error messages for each field. If a field is invalid, the error message is appended to the list. Once all the fields have been validated, if there are any error messages in the list, the `div` containing the error messages is formatted and printed at the top of the form. Here's the script:

[Click here to view code image](#)

```
$(function() {
  $("#registrationForm").submit(function (event) {
    var errors = [];

    if (!$(this.name).val()) {
      errors.push("<li>You must enter a name.</li>");
      $(this.name).parents("div.required").addClass("error");
      event.preventDefault();
    }
  });
});
```

```

    if (!$("input[name='gender']:checked", this).length) {
        errors.push("<li>You must select a gender.</li>");
        $("input[name='gender']",
this).parents("div.required").addClass("error");
        event.preventDefault();
    }

    if (errors.length > 0) {
        $(this).prepend("<div class='errors'>You must "
            + "correct the following errors:<ul>"
            + errors.join("") + "</ul></div>");
    }
});
});

```

If you include this code in your page and test it, you'll see that when you submit an invalid form, it properly marks the labels of invalid fields, prints the error message at the top form, and prevents the form from being submitted. However, one bug should be fixed. What happens when you submit the form with invalid input a second time? As you'll see, a second set of error messages will be printed at the top of the page. To fix this bug, I just need to add the following line to the event handler:

[Click here to view code image](#)

```

$("div.errors", this).remove();

```

I added it right before the `if` statement that checks whether to print the error messages. It uses the jQuery `remove()` method to remove any elements that match the selector.

The final form with all of the error messages for invalid input displayed appears in [Figure 19.5](#).

The screenshot shows a web browser window titled "Registration Form". The address bar displays a file path: `file:///Users/rcolburn/Dropbox/tywebpub_6e/examples/18...`. The page content includes a heading "Registration Form" and a paragraph: "Please fill out the form below to register for our site. Fields with bold labels are required." Below this, a red-bordered box contains the message: "You must correct the following errors:" followed by a bulleted list: "• You must enter a name." and "• You must select a gender." The form fields are: "Name" (a green text input field), "Gender" (radio buttons for "male" and "female"), "Operating System" (a dropdown menu showing "Windows"), "Toys" (checkboxes for "Digital Camera", "MP3 Player", and "Wireless LAN"), "Portrait" (a "Choose File" button and the text "No file chosen"), and "Mini Biography" (a large green text area). At the bottom is a "register" button.

FIGURE 19.5 An alert message for multiple fields.

Hiding and Showing Content

One way to help users deal with a lot of information presented on a single page is to hide some of it when the page loads and to provide controls to let them view the information that interests them. In this example, I create a frequently asked questions page. On this page, the questions will be organized in sections. Each section will start out collapsed, and the answers to the questions will be hidden as well. Users can display the answers by clicking the corresponding questions.

In this example, I apply an approach referred to as unobtrusive JavaScript. The philosophy behind it is that the behavior added by JavaScript should be clearly separated from the presentation applied using HTML and *Cascading Style Sheets* (CSS). The page should work and be presentable without JavaScript, and JavaScript code should not be mixed

with the markup on the page.

In practice, what this means is that event handlers should be specified in scripts instead of HTML attributes. Also, in this example, using JavaScript unobtrusively means that if the user has JavaScript turned off, she will see all the questions and answers by default, rather than seeing a page with the questions hidden and no means of displaying them.

In the previous lesson, you learned how to hide and show content using jQuery. In this example, I'm going to explain how to accomplish the same thing using plain JavaScript, and then provide the jQuery version as a comparison. jQuery is a convenience, but it's not necessary for building things with JavaScript, as you'll see.

▼ Exercise 19.2: Hiding and Showing Content

The exercise starts with a web page that doesn't include JavaScript and that displays the frequently asked questions. Here's the source for the page. [Figure 19.6](#) shows the page in a browser.

Input ▼

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
  <title>Frequently Asked Questions</title>
  <style type="text/css" media="screen">
    dt { margin-bottom: .5em; font-size: 125%;}
    dd { margin-bottom: 1em;}
  </style>
</head>
<body>
<h1>Frequently Asked Questions</h1>
<dl id="faq">
  <dt>Don't I need a development environment to work with JavaScript?
</dt>
  <dd>Nope. As with HTML, all you need is a text editor and a browser
that sup-
ports JavaScript. You might be confusing JavaScript with Java, a more
comprehen-
sive programming language that needs at least a compiler for its programs
to run.
However, tools like FireBug for Firefox, the Internet Explorer Developer
Toolbar,
and Safari's Web Inspector can make your life easier. Consult the
documentation
on those tools to learn more about their JavaScript features.</dd>
  <dt>What is AJAX?</dt>
  <dd>One topic we haven't covered yet is AJAX. AJAX is a term used to
describe
scripts that communicate with the server without requiring a Web page to
be
fully reloaded. For example, you can use it to fetch information and
display it
on the page, or to submit a form for processing, all without changing the
full
page in the browser. I'll discuss AJAX in detail in Lesson 16, "Using
JavaScript
```

```

Libraries."</dd>
    <dt>When I use JavaScript, do I need to accommodate users whose
browsers may
not support JavaScript or who have disabled it?</dt>
    <dd>Some estimates indicate that over 90% of Web users have JavaScript
enabled. However, unless you have a really good reason not to, you should
make
accommodations for users without JavaScript. You need not offer users who
don't
have JavaScript an identical experience to those who have it, but they
should be
able to access your site. For example, if you run an online store, do you
really
want to shut out users because of their browser configuration? </dd>
</dl>
</body>
</html>

```

Output ▼

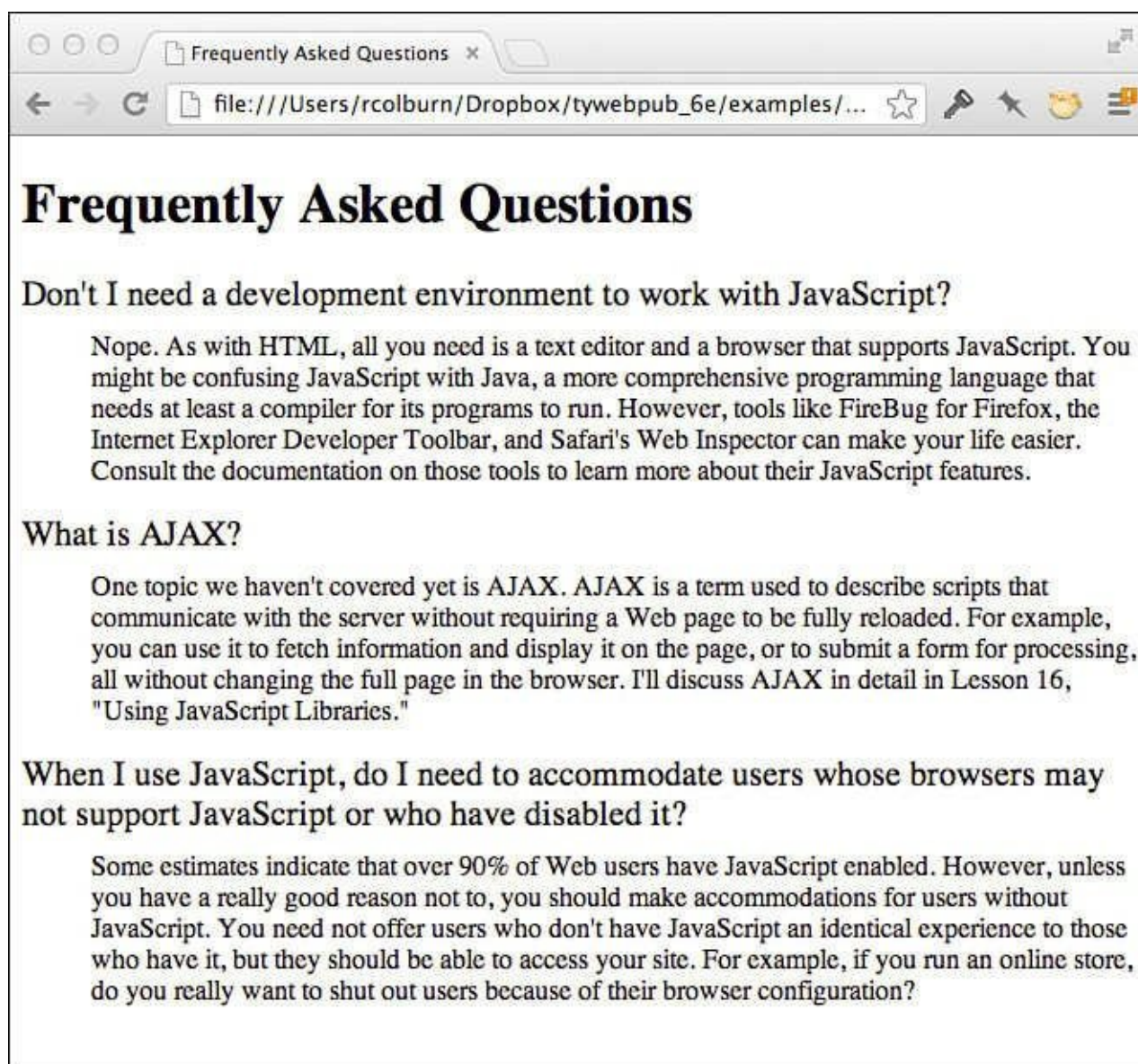


FIGURE 19.6 The FAQ page.

The page is designed so that it works perfectly well without JavaScript; all the

questions and answers are displayed so that the user can read them. This is what unobtrusive JavaScript is all about.

Adding the Script After the page has been created and you've confirmed that it's working correctly, the next step is to add JavaScript into the mix. To include the script on the page, I just need to add the link to the external script to the header:

[Click here to view code image](#)

```
<script type="text/javascript" src="faq.js"></script>
```

That `<script>` tag loads and executes the script in the file `faq.js`. After the JavaScript has been added, the answers to the questions are hidden, as shown in [Figure 19.7](#).



FIGURE 19.7 The FAQ page with the JavaScript included.

Here's the JavaScript contained in the `faq.js` file:

[Click here to view code image](#)

```
window.onload = function() {  
    var faqList, answers, questionLinks, questions, currentNode, i, j;  
  
    faqList = document.getElementById("faq");  
    answers = faqList.getElementsByTagName("dd");  
  
    for (i = 0; i < answers.length; i++) {  
        answers[i].style.display = 'none';  
    }  
  
    questions = faqList.getElementsByTagName("dt");  
  
    for (i = 0; i < questions.length; i++) {  
        questions[i].onclick = function() {  
            currentNode = this.nextSibling;  
            while (currentNode) {  
                if (currentNode.nodeType == "1" && currentNode.tagName ==  
"DD") {  
                    if (currentNode.style.display == 'none') {  
                        currentNode.style.display = 'block';  
                    }  
                }  
                currentNode = currentNode.nextSibling;  
            }  
        }  
    }  
}
```

```

        }
        else {
            currentNode.style.display = 'none';
        }

        break;
    }

    currentNode = currentNode.nextSibling;
}

return false;
};
}
}

```

This JavaScript code is significantly more complex than any used previously in the book. Take a look at the first line, which is repeated here:

```

window.onload = function() {

```

This is where the unobtrusiveness comes in. Instead of calling a function using the `onload` attribute of the `<body>` tag to start up the JavaScript for the page, I assign an anonymous function to the `onload` property of the window object. The code inside the function will run when the `onload` event for the window is fired by the browser. Setting up my JavaScript this way allows me to include this JavaScript on any page without modifying the markup to bind it to an event. That's handled here.

This is the method for binding functions to events programmatically. Each element has properties for the events it supports. To bind an event handler to them, you assign the function to that property. You can do so by declaring an anonymous function in the assignment statement, as I did in this example, or you can assign the function by name, like this:

```

function doStuff() {
    // Does stuff
}
window.onload = doStuff;

```

In this case, I intentionally left the parentheses out when I used the function name. That's because I'm assigning the function itself to the `onload` property, as opposed to assigning the value returned by `doStuff()` to that property.

Note

When you declare an anonymous function in an assignment statement, you must make sure to include the semicolon after the closing brace. Normally when you declare functions, a semicolon is not needed, but because the function declaration is part of the assignment statement, that statement has to be terminated with a semicolon or you'll get a syntax error when the browser tries to interpret the JavaScript.

On the next line, I declare all the variables I use in this function. JavaScript is a bit

different from many other languages in that variables cannot have “block” scope. For example, in most languages, if you declare a variable inside the body of an `if` statement, that variable will go away once the `if` statement is finished. Not so in JavaScript. A variable declared anywhere inside a function will be accessible from that point onward in the function, regardless of where it was declared. For that reason, declaring all your variables at the top of the function is one way to avoid confusing bugs.

Looking Up Elements in the Document The preceding lesson discussed the `document` object a little bit and mentioned that it provides access to the full contents of the web page. The representation of the page that is accessible via JavaScript is referred to as the *Document Object Model*, or DOM. The entire page is represented as a tree, starting at the root element, represented by the `<html>` tag. If you leave out the `<html>` tag, the browser will add it to the DOM when it renders the page. The DOM for this page is shown in [Figure 19.8](#).

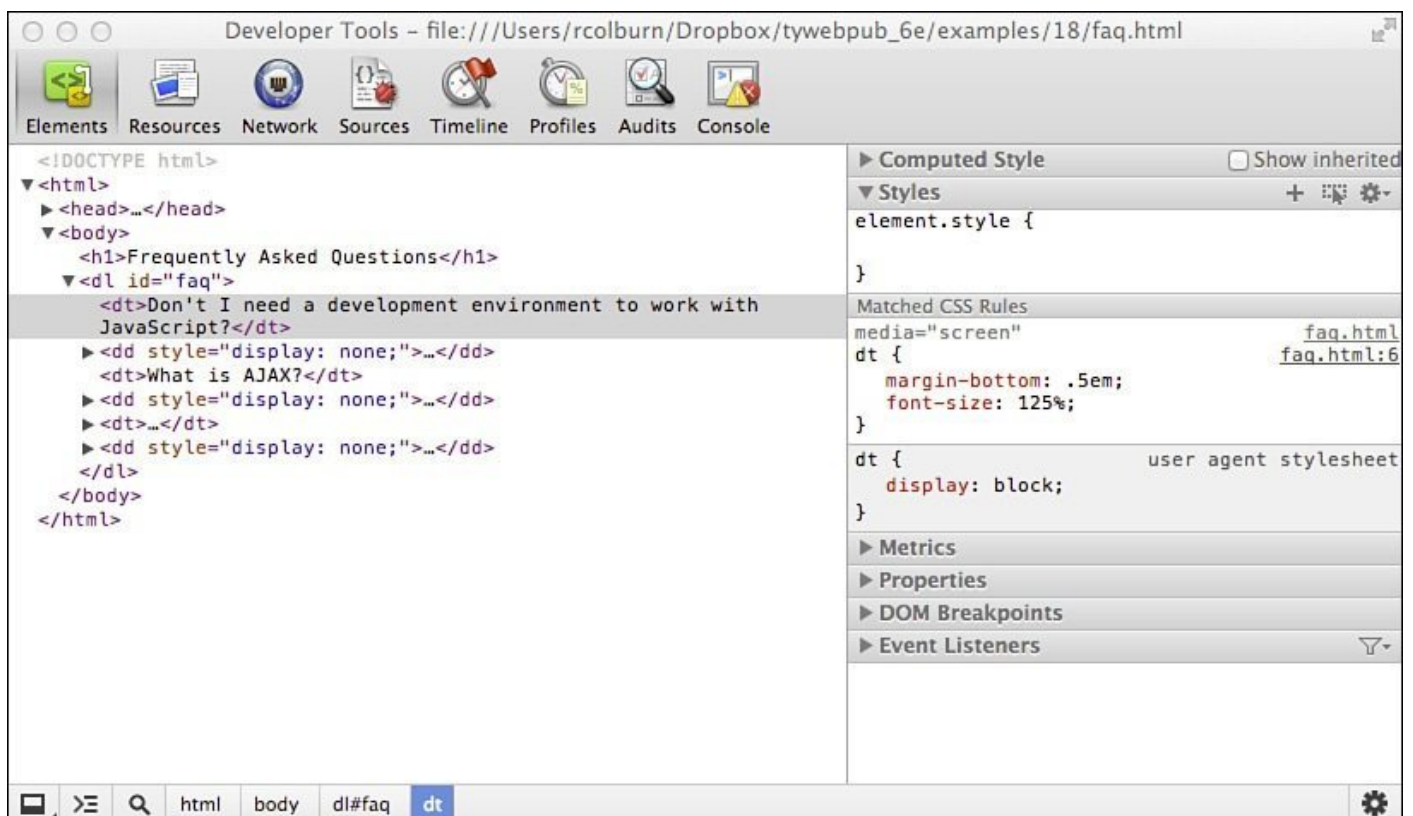


FIGURE 19.8 The DOM for the FAQ page, shown in the Chrome Developer Tools.

There are a number of ways to dig into the DOM. The browser provides access to the parent of each element, as well as its siblings and children, so you can reach any element that way. However, navigating your way to elements in the page that way is tedious. Fortunately, there are some shortcuts available.

These shortcuts, methods that can be called on the `document` object, are listed in [Table 19.1](#).

Method	Description
<code>getElementsByTagName (name)</code>	Retrieves a list of elements with the supplied tag name. This can also be called on a specific element, and it will return a list of the descendants of that element with the specified tag name.
<code>getElementById(id)</code>	Retrieves the element with the specified ID. IDs are assigned using the <code>id</code> attribute. This is one of the areas in which JavaScript intersects with CSS.
<code>getElementByName (name)</code>	Retrieves elements with the specified value as their <code>name</code> attribute. Usually used with forms or form fields, both of which use the <code>name</code> attribute.

TABLE 19.1 Methods for Accessing the DOM

To set up the expanding and collapsing properly, I must hide the answers to the questions and bind an event to the questions that expands them when users click them. First, I need to look up the elements I want to modify in the DOM.

[Click here to view code image](#)

```
faqList = document.getElementById("faq");
answers = faqList.getElementsByTagName("dd");
```

The first line gets the element with the ID `faq`. That's the ID I assigned to my definition list in the markup. Then the next line returns a list of all the `dd` elements that are children of the element now assigned to `faqList`. I could skip the step of looking up the `faq` list first, but then if this page included multiple definition lists, the behavior would be applied to all of them rather than just the `faq`. This is also a useful precaution in case this JavaScript file is included on more than one page. In the end, I have a list of `dd` elements.

Changing Styles I grabbed the list of `dd` elements so that they can be hidden when the page loads. I could have hidden them using a style sheet or the style attribute of each of the `dd` elements, but that wouldn't be unobtrusive. If a user without JavaScript visited the page, the answers to the questions would be hidden and there wouldn't be any way to reveal the answers. It's better to hide them with JavaScript.

There are two ways to hide elements with CSS: you can set the `display` property to `none` or the `visibility` property to `hidden`. Using the `display` property will hide the element completely and remove it from the DOM. The `visibility` property hides the content in the element but leaves the space it takes up empty. So for this case, using the `display` property makes more sense. Every element in the document has a `style` property, and that property has its own properties for each CSS property. Here's the code that hides each of the `dd` elements:

[Click here to view code image](#)

```
for (i = 0; i < answers.length; i++) {
    answers[i].style.display = 'none';
}
```

The `for` loop iterates over each of the elements, and inside the loop, I set the `display` property to `none`. Once the page loads, the answers will be hidden.

Traversing the Document The final step is to bind the event that toggles the display of the answers to each of the questions. This is the most complex bit of code on the page. First, let me explain how the event handler works:

[Click here to view code image](#)

```
function() {
    currentNode = this.nextSibling;
    while (currentNode) {
        if (currentNode.nodeType == "1" && currentNode.tagName == "DD") {
            if (currentNode.style.display == 'none') {
                currentNode.style.display = 'block';
            }
            else {
                currentNode.style.display = 'none';
            }
            break;
        }
        currentNode = currentNode.nextSibling;
    }
    return false;
};
```

That's the function that will be used as the `onclick` handler for each of the questions. As you may remember, in the context of an event handler, `this` is the element associated with the event. The main challenge in this function is locating the answer associated with the question the user clicked on, and displaying it.

To do so, the function will navigate through the DOM to find the next DD element in the DOM tree following the DT element that the user clicked on. First, I use the `nextSibling` property of `this`, and then I start a `while` loop that will iterate over each of the siblings of that element. The `while` condition ensures that the loop will run until `this` runs out of siblings.

The `nextSibling` property is a reference to the next node in the DOM tree. A node is different from an element. HTML elements are nodes, but the whitespace between tags is a node, as is the text inside a tag. So the `nextSibling` of a node might very well be the return character at the end of the line following the tag. There are a number of other properties associated with nodes as well that can be used to traverse the document. Some are listed in [Table 19.2](#).

Method	Description
<code>childNodes</code>	An array of all the children of a node.
<code>firstChild</code>	The first child node of a node.
<code>innerHTML</code>	The markup and content inside a node. You can set this property to change the contents of a node.
<code>lastChild</code>	The last child of a node.
<code>nextSibling</code>	The next sibling of the node (at the same level of the DOM tree).
<code>parentNode</code>	The parent of the current node.
<code>previousSibling</code>	The node that precedes the current node at the same level of the tree.

TABLE 19.2 Node Properties for Navigating the DOM

All the properties in the table are `null` if it's not possible to traverse the DOM in that direction. For example, if a node has no child nodes, its `lastChild` property will be `null`.

Here's what happens when a user clicks one of the questions. As mentioned, a `while` loop will iterate over the siblings of the question. Inside the `while` loop, I check the `nodeType` and `tagName` of the current node.

The `nodeType` property contains a number that identifies what type of node is being processed. Element nodes have a node type of 1. Attributes are node type 2, and text nodes are type 3. There are 12 total node types, but those three are the main ones you'll use. In this function, I'm searching for the `<dd>` tag that follows the `DT` tag that contains the question. I have to check the node type before checking the `tagName` property because only elements (that have node type 1) support the `tagName` property. If I didn't check the node type first, other node types would cause errors.

Each sibling node that follows the original `<dt>` is tested, and as soon as a `<dd>` element is found, the script toggles the visibility of that element. It then uses the `break` statement to stop executing the loop. If the node is not a `<dd>` element, then the next sibling of `currentNode` is assigned to the `currentNode` variable, and the loop is executed again. If the `<dd>` element is never found, then when there are no more siblings, the `currentNode` variable will be set to `null`, and execution of the loop will stop.

At the end, the function returns `false`:

[Click here to view code image](#)

```
questions = faqList.getElementsByTagName("dt");
for (i = 0; i < questions.length; i++) {
    questions[i].onclick = function() {
        // The actual event handling code goes here.
    }
}
```

First, I use `getElementsByTagName()` to get a list of all the `<dt>` tags that are children of `faqList`. Then I use a `for` loop to iterate over them and bind the function described previously to their `onclick` event.

The Same Code with jQuery

Now here's a look at code that achieves the same results using jQuery. There are some flaws in the previous code as well that this solution gets around. For example, for reasons of simplicity and cross-browser compatibility, I specified the click event handler, attaching it directly to the `onclick` property of the relevant element. I did the same thing with the `body.onload` handler. As you know, this approach can interfere with other scripts you might want to use on the page.

First I load jQuery from the CDN with this line of HTML:

[Click here to view code image](#)

```
<script src="https://code.jquery.com/jquery-2.1.4.min.js"></script>
```

Here's the new `faq.js` script:

[Click here to view code image](#)

```
$(function() {  
    $("#faq dd").hide();  
    $("#faq dt").click(function(event) {  
        event.preventDefault();  
        $(this).next("dd").toggle();  
    });  
});
```

As you can see, the libraries enable me to accomplish a lot more than I could writing raw JavaScript with just a few lines of code.

Adding New Content to a Page

The last example demonstrated how to modify styles on a page. In this example, I explain how to modify the content on a page using JavaScript. You can create new elements in JavaScript and then attach them to the document in any location that you choose. You can also modify elements that are already on the page or remove elements if you need to do so.

▼ Exercise 19.3: Add an Expand All/Collapse All Link to the FAQ

In this example, I'll be adding a new feature to the FAQ page presented in the previous example. In that example, I illustrated how to add new features to a page using JavaScript without modifying the markup in any way. This example will continue along those lines. In fact, I won't be making any changes to the markup on the page; all the changes will take place inside the JavaScript file.

In this example, I add a link to the page that expands all the questions in the FAQ, or, if all the questions are already expanded, will collapse all the questions. The label on the link will change depending on its behavior, and the function of the link

will change if the user individually collapses or expands all the questions.

Adding the Link to the Page Because the link only functions if the user has JavaScript enabled, I am going to add it dynamically using JavaScript. I've added a new function to the JavaScript file that takes care of adding the link, which I call from the `onload` handler for the page. The function adds more than just a link to the page. It adds a link, a `<div>` containing the link, and the `onclick` handler for the link. Here's the function, which I've named `addExpandAllLink()`:

[Click here to view code image](#)

```
function addExpandAllLink() {
    var expandAllDiv, expandAllLink, faq;

    expandAllDiv = document.createElement("div");
    expandAllDiv.setAttribute("id", "expandAll");

    expandAllLink = document.createElement("a");
    expandAllLink.setAttribute("href", "#");
    expandAllLink.setAttribute("id", "expandAllLink");
    expandAllLink.appendChild(document.createTextNode("Expand All"));

    expandAllDiv.appendChild(expandAllLink);

    expandAllLink.onclick = function() {
        var faqList, answers;
        faqList = document.getElementById("faq");
        answers = faqList.getElementsByTagName("dd");

        if (this.innerHTML == "Expand All") {
            for (i = 0; i < answers.length; i++) {
                answers[i].style.display = 'block';
            }
            this.innerHTML = "Collapse All";
        }
        else {
            for (i = 0; i < answers.length; i++) {
                answers[i].style.display = 'none';
            }
            this.innerHTML = "Expand All";
        }
        return false;
    };

    faq = document.getElementById("faq");
    faq.insertBefore(expandAllDiv, faq.firstChild);
}
```

First, I declare the variables I use in the function, and then I start creating the elements. The `createElement()` method of the `document` object is used to create an element. It accepts the element name as the argument. I create the `<div>` element and then call the `setAttribute()` method to add the `id` attribute to that element. The `setAttribute()` method takes two arguments: the attribute name and the value for that attribute. Then I create the link by creating a new `<a>` element. I set the `href` attribute to `#`, because the event handler for the link's `onclick` event will return `false` anyway, and I add an `id` for the link, too. To

add the link text, I call the `document.createTextNode()` method:

[Click here to view code image](#)

```
expandAllLink.appendChild(document.createTextNode("Expand All"));
```

I pass the results of that method call to the `appendChild()` method of `expandAllLink`, which results in the text node being placed inside the `<a>` tag. Then on the next line I append the link to the `<div>`, again using `appendChild()`. The last thing to do before appending the `<div>` to an element that's already on the page (causing it to appear) is to add the `onclick` handler to the link.

I'm again attaching the `onclick` handler using an anonymous function, as I did in the previous example. In this case, I use the same technique I did in the previous example, obtaining a reference to the `<div>` with the ID `faq` and then retrieving a list of `<dd>` elements inside it.

At that point, I inspect the contents of `this.innerHTML`. In an event handler, `this` is a reference to the element upon which the event was called, so in this case, it's the link. The `innerHTML` property contains whatever is inside that element—in this case, the link text. If the link text is "Expand All," I iterate over each of the answers and set their `display` property to `block`. Then I modify the `this.innerHTML` to read "Collapse All". That changes the link text to `Collapse All`, which not only alters the display but causes the same function to hide all the answers when the user clicks on the link again. Then the function returns `false` so that the link itself is not processed.

Once the `onclick` handler is set up, I add the link to the document. I want to insert the link immediately before the list of frequently asked questions. To do so, I get a reference to its `<div>` using `getElementById()` and then use `insertBefore()` to put it in the right place:

[Click here to view code image](#)

```
faq = document.getElementById("faq");  
faq.insertBefore(expandAllDiv, faq.firstChild);
```

[Table 19.3](#) contains a list of methods that can be used to modify the document. All of them are methods of elements.

Method	Description
<code>appendChild(element)</code>	Adds the element to the page as a child of the method's target
<code>insertBefore(new, ref)</code>	Inserts the element <code>new</code> before the element <code>ref</code> on the list of children of the method's target
<code>removeAttribute(name)</code>	Removes the attribute with the supplied name from the method's target
<code>removeChild(element)</code>	Removes the child of the method's target passed in as an argument
<code>replaceChild(inserted, replaced)</code>	Replaces the child element of the method's target passed as the <code>inserted</code> argument with the element passed as the parameter <code>replaced</code>
<code>setAttribute(name, value)</code>	Sets an attribute on the method target with the name and value passed in as arguments

TABLE 19.3 Methods for Accessing the DOM

There's one other big change I made to the scripts for the page. I added a call to a new function in the handler for the click event for the questions on the page:

```
updateExpandAllLink();
```

That's a call to a new function I wrote, which switches the `Expand All / Collapse All` link if the user manually collapses or expands all the questions. When the page is opened, all the questions are collapsed, and the link expands them all. After the user has expanded them all one at a time, this function will switch the link to `Collapse All`. The function is called every time the user clicks on a question. It inspects the answers to determine whether they are all collapsed or all expanded, and it adjusts the link text accordingly. Here's the source for that function:

[Click here to view code image](#)

```
function updateExpandAllLink() {
  var faqList, answers, expandAllLink, switchLink;

  faqList = document.getElementById("faq");
  answers = faqList.getElementsByTagName("dd");
  expandAllLink = document.getElementById("expandAllLink");
  switchLink = true;

  if (expandAllLink.innerHTML == "Expand All") {
    for (i = 0; i < answers.length; i++) {
      if (answers[i].style.display == 'none') {
        switchLink = false;
      }
    }

    if (switchLink) {
      expandAllLink.innerHTML = "Collapse All";
    }
  }
```

```

    }
  }
  else {
    for (i = 0; i < answers.length; i++) {
      if (answers[i].style.display == 'block') {
        switchLink = false;
      }
    }

    if (switchLink) {
      expandAllLink.innerHTML = "Expand All";
    }
  }
}

```

This function starts with some setup. I declare the variables I will be using and retrieve the elements I need to access from the DOM. I also set the variable `switchLink` to `true`. This variable is used to track whether I need to switch the link text in the `Expand All` link. Once everything is set up, I use an `if` statement to test the state of the link. If the link text is set to `Expand All`, it checks each of the answers. If any of them are hidden, it leaves the link as is. If all of them are displayed, it changes the link text to `Collapse All`. If the link text is already `Collapse All`, the test is the opposite. It switches the link text to `Expand All` if all the questions are hidden.

Now let's look at how I could accomplish the same thing using jQuery. Inside the main ready function, I add the following code:

[Click here to view code image](#)

```

$("<a href='#' id='expandAll'>Expand All</a>").insertAfter("h1");

$("#expandAll").click(function (event) {
  event.preventDefault();

  if ($(this).html() == "Expand All") {
    $("#faq dd").show();
    $(this).html("Collapse All");
  } else {
    $("#faq dd").hide();
    $(this).html("Expand All");
  }
});

```

First, I add the new link to the page. I use the jQuery `insertAfter()` method, which adds the content specified after the selector passed to it. Had the selector matched multiple elements, the content would have been appended after each of them.

After I add the new element to the page, I add a click handler for the link. It performs the same functions as does the non-jQuery version I presented previously, but it uses jQuery methods to accomplish the same thing with less code.

Summary

This lesson demonstrated a number of common tasks associated with programming in JavaScript. It illustrated how to access the values in forms and check them for errors. It also explained how you can manipulate the styles on a page and even the contents of a page using JavaScript. The final two examples were written in a style referred to as unobtrusive JavaScript, which involves writing JavaScript in such a way that the page still works even if the user has disabled JavaScript or his browser does not offer JavaScript support. JavaScript is used to enhance the user's experience, but the functionality of the page is not dependent on JavaScript. This approach is generally favored as the preferable way to write JavaScript these days. It separates JavaScript code from the markup on the page and ensures support for the largest number of users, including users with mobile browsers that may not support JavaScript functionality.

Workshop

The following workshop includes questions, a quiz, and exercises related to the uses of JavaScript.

Q&A

Q Can you point me in the direction of more scripts that I can integrate with my pages?

A A number of sites provide JavaScript you can copy and paste for use on your own pages, but this approach has fallen out of favor. Given the simplicity of building functionality with jQuery and other libraries, the best bet is to find libraries that provide the building blocks you need to write your own scripts and then put the pieces together to accomplish your goals.

Q In what cases might you want to use JavaScript without a library like JavaScript?

A If your JavaScript is very simple, it may be worth it to leave out supporting libraries. But for most web developers, starting with a tool that accounts for differences between browsers is a great way to avoid bugs and get things done quickly and simply. It's generally better to make use of these sorts of libraries to write JavaScript from scratch. It's a good idea to know how to write JavaScript without libraries, but using them almost always makes sense.

Quiz

1. What happens whenever a user clicks a link, button, or form element on a web page?
2. In an event handler, what does `this` refer to?
3. What kinds of nodes on a page can be associated with properties like `nextChild` and `previousChild`?
4. How does form validation with JavaScript conserve server resources?

Quiz Answers

1. Whenever a user clicks a link, a button, or any form element, the browser generates an event signal that can be captured by one of the event handlers mentioned in the previous lesson.
2. In event handlers, `this` is a reference to the element on which the event was called. So in an event handler for the `onclick` event of a link, `this` would refer to the link that the user clicked on.
3. Nodes in the DOM can include HTML elements, text inside HTML elements, and even whitespace between elements.
4. JavaScript enables you to do error checking in forms on the browser side before the form is ever submitted to the server. A script must access the server before it can determine the validity of the entries on a form. (Note that even if you use JavaScript form validation, you must validate user input on the server, too, because users can bypass the JavaScript if they choose.)

Exercises

1. Change the HTML validation example to add error messages to the page above the form when validation fails.
2. Add a Preview button to the form validation example that displays the values the user entered below the form.
3. Modify the FAQ example so that users can click a link for each question to remove that question from the page entirely.

Lesson 20. Working with Frames and Linked Windows

In the early days of the Web, two significant limitations of web browsers were that they could only display one document in a browser window at a time and that sites couldn't open more browser windows if needed. The Web has evolved to enable developers to combine web pages in browser windows in all sorts of ways. Inline frames enable you to embed web pages within other pages in the same way you would an image, and linked windows enable you to create links that open pages in new browser windows. These tools offer a lot of power, but improper usage can make your sites far less usable. It's important to learn not only how to use them, but when to use them. You'll learn all about the following topics:

- What inline frames are and how they can affect your layout
- How to work with linked windows
- What happened to frames

What Are Frames?

HTML5 enables you to embed a second document into an existing page using what's called an *inline frame*. Inline frames are like images or video players; they can be included anywhere within a page. Over time, inline frames became more commonly used, and frames created using framesets were made obsolete. HTML5 does not include support for frames created using framesets, leaving only inline frames.

Why Were Frames Removed from HTML5?

Frames were removed from the HTML specification for several reasons. The primary reason that they were removed was because the `<frameset>` tag replaces the `<body>` tag. This can cause problems with some assistive technology as they are built to expect a body on any web page.

But there are other reasons why frames caused problems. Frames prevent customers from correctly bookmarking web pages. You visit a frame site and then click some links, but when you try to bookmark the internal pages, your bookmark records only the top-level page. Some designers tried to solve this problem with JavaScript, but that simply added to the complexity of the pages and made them even more difficult for screen readers to use.

Printing pages in frames is also difficult. Although most browsers will print the first page or screen, they struggle with getting all the content. Plus, if the frameset doesn't fit on the page, it can result in some strange use of paper.

Linking to frames from external sites can be difficult. Often you have the choice of linking to the front page or to an internal framed page that is missing navigation and other elements contained in the frameset. This means that a framed page that is referenced in search engine results could end up being displayed with no navigation or any site references. This is not a good user experience, and most designers want their site logo and navigation to appear on every page.

What About Iframes?

Iframes are different from frames because they are simply embedded into the web page, just like an image or other multimedia. Iframes add all the benefits of frames with none of the drawbacks. You can embed content from other parts of your website or completely different websites right inside your web pages.

HTML5 adds a few new features to the `<iframe>` tag to add security and flexibility to your frames. I will discuss those features later in this lesson.

Working with Linked Windows

Before you learn how to use iframes, you need to learn about the `target` attribute of the `<a>` tag. It enables you to direct a link to open a page in an inline frame or a new window. This attribute takes the following form:

```
target="browsing_context"
```

To drop a bit of jargon on you, the value of the `target` attribute is the name of a browsing context. Usually, when you click a hyperlink, the page to which you're linking replaces the current page in the browser window. When you use the `target` attribute, you can open links in new windows, or in existing windows other than the one that the link is in. With frames, you can use the `target` attribute to display the linked page in a different frame. There are also some keywords you can use that represent browsing contexts by function rather than name.

When you use the `target` attribute inside an `<a>` tag, the browser first checks whether a browsing context with the name that matches the value of the attribute exists. If it does, the document pointed to by the hyperlink replaces the current contents of that browsing context. On the other hand, if the named browsing context does not exist, a new browser window or tab opens with that name assigned to it. Then the document pointed to by the hyperlink is loaded into the newly created window.

Browsing Context Keywords

The `target` attribute works not only with window names but also with keywords that represent a type of context. All of these keywords are case sensitive and begin with an underscore (`_`) character. They enable you to target links at specific browsing context relative to the current browsing context.

[Table 20.1](#) lists the browsing context keywords and describes their use.

target Name	Description
target="_blank"	Forces the document referenced by the <a> tag to be loaded into a new unnamed window.
target="_self"	Causes the document referenced by the <a> tag to be loaded into the current browsing context—the window or frame that contains the <a> tag. This is the default behavior of links.
target="_parent"	Specifically for use with frames. Forces the link to load into the parent of the current browsing context. If the current document has no parent, however, target="_self" will be used.
target="_top"	Forces the link to load into the full web browser window, replacing the current page entirely. If the current document is already at the top, however, target="_self" will be used. More often than not, when you create links to other sites on the Web, you don't want them to open within a frame. Adding target="_top" to the link will prevent this from occurring.

TABLE 20.1 Magic target Names

▼ Exercise 20.1: Working with Windows

Each of the hyperlinks in the following exercise uses the `target` attribute to open a web page in a different browser window. The concepts you'll learn here will help you understand later how targeted hyperlinks work with frames.

In this exercise, you create four separate HTML documents that use hyperlinks, including the `target` attribute. You use these hyperlinks to open two new windows, called `yellow_page` and `blue_page`, as shown in [Figure 20.1](#). The top window is the original web browser window (the red page), `yellow_page` is at the bottom right, and `blue_page` is at the bottom left.

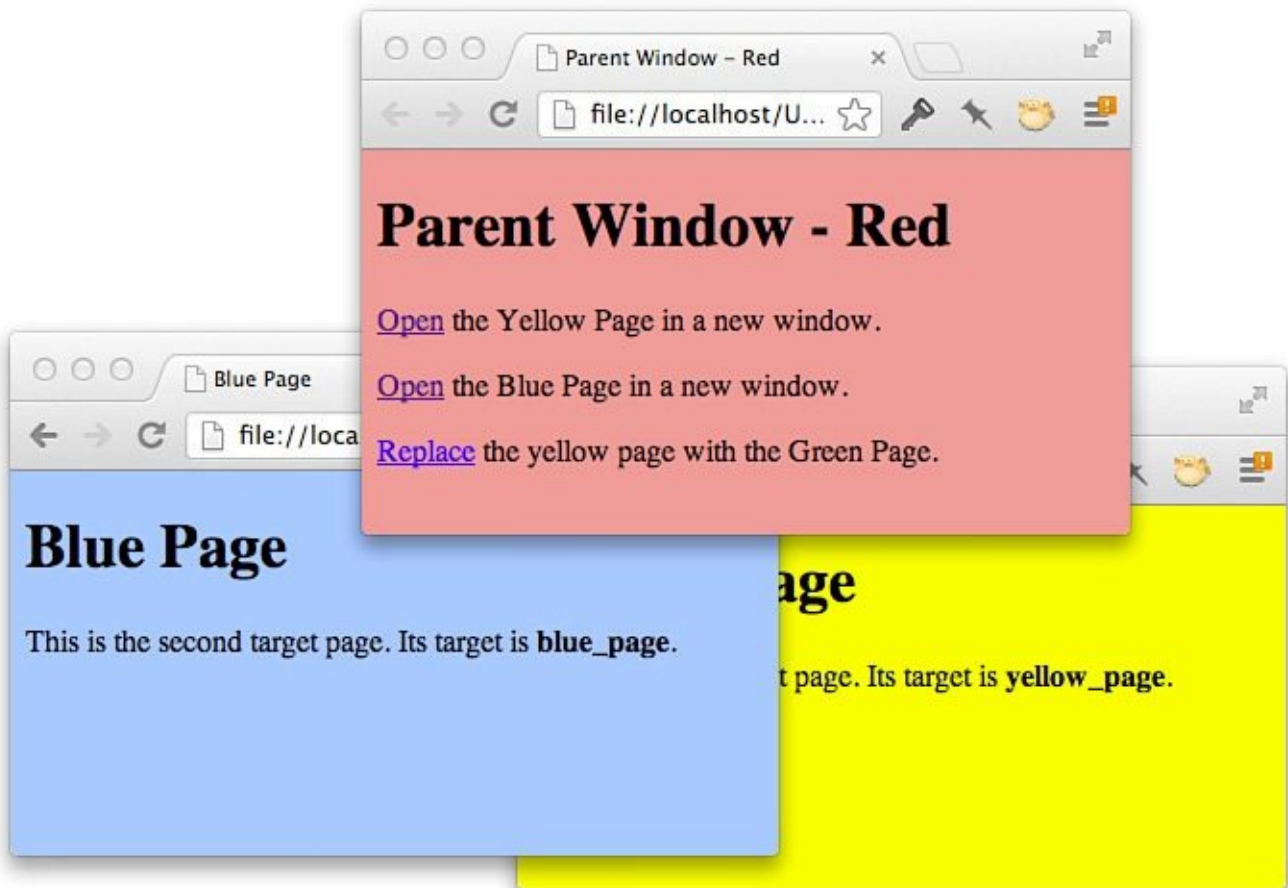


FIGURE 20.1 Using the `target` attribute indicates that links should open new windows.

First, create the document to be displayed by the main web browser window, shown in [Figure 20.2](#), by opening your text editor of choice and entering the following lines of code:

Input ▼

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
<title>Parent Window - Red</title>
</head>
<body style="background-color: #ff9999;">
  <h1>Parent Window - Red</h1>
  <p><a href="yellow.html" target="yellow_page">Open</a> the Yellow Page
in a
new window.</p>
  <p><a href="blue.html" target="blue_page">Open</a> the Blue Page in a
new
window.</p>
  <p><a href="green.html" target="yellow_page">Replace</a> the yellow
page with
the Green Page.</p>
</body>
</html>
```

Output ▼

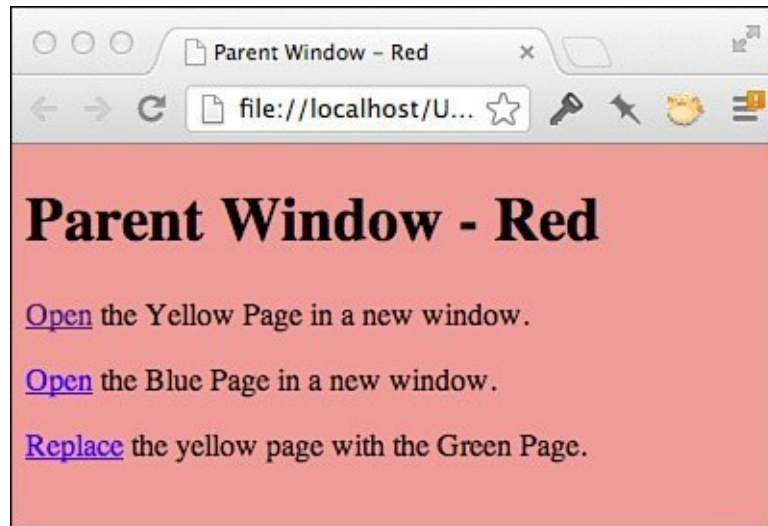


FIGURE 20.2 The parent window (the red page).

This creates a light-red page that links to the other three pages. Save this HTML source as `parent.html`.

Next, create a document called `yellow.html` (see [Figure 20.3](#)) by entering the following code:

Input ▼

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
<title>Yellow Page</title>
</head>
<body style="background-color: #ffff33;">
  <h1>Yellow Page</h1>
  <p>This is the first target page. Its target is <b>yellow_page</b>.
</p>
</body>
</html>
```

Output ▼



FIGURE 20.3 `yellow.html` web browser window named `yellow_page`.

After saving `yellow.html`, create another document called `blue.html` (see [Figure 20.4](#)) by entering the following code:

Input ▼

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
<title>Blue Page</title>
</head>
<body style="background-color: #99ccff;">
  <h1>Blue Page</h1>
  <p>This is the second target page. Its target is <b>blue_page</b>.</p>
</body>
</html>
```

Output ▼

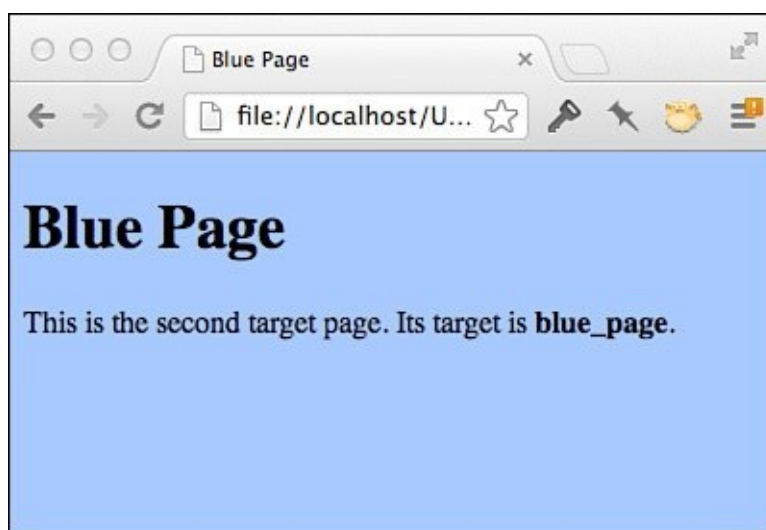


FIGURE 20.4 `blue.html` displayed in the web browser window named `blue_window`.

Next, create a fourth document called `green.html`, which looks like the

following:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
<title>Green Page</title>
</head>
<body style="background-color: #ccffcc;">
  <h1>Green Page</h1>
  <p>This is the third target page. Its target is <b>yellow_page</b>. It
should
  replace the yellow page in the browser.</p>
</body>
</html>
```

To complete the exercise, load `parent.html` (the red page) into your web browser. Click the first hyperlink to open the yellow page in a second browser window. This happens because the first hyperlink contains the attribute `target="yellow_page"`, as the following code from `parent.html` demonstrates:

[Click here to view code image](#)

```
<p><a href="yellow.html" target="yellow_page">Open</a> the Yellow Page in a
new
window.</p>
```

Now return to the red page and click the second link. The blue page opens in a third browser window. Note that the new windows won't be laid out like the ones shown in [Figure 20.1](#); modern browsers open new windows as new tabs. The following `target="blue_page"` statement in the `parent.html` page is what causes the new window to open:

[Click here to view code image](#)

```
<a href="blue.html" target="blue_page">Open</a> the Blue Page in a new
window.
</p>
```

The previous two examples opened each of the web pages in a new browser window. The third link, however, uses the `target="yellow_page"` statement to open the green page in the window named `yellow_page`. You accomplish this using the following code in `parent.html`:

[Click here to view code image](#)

```
<p><a href="green.html" target="yellow_page">Replace</a> the yellow page
with the Green Page.</p>
```

Because you already opened the `yellow_page` window when you clicked the link for the yellow page, the green page should replace the page that's already in it. To verify this, click the third hyperlink on the red page. This replaces the contents of the yellow page (with the `yellow_page` target name) with the green page (`green.html`), as shown in [Figure 20.5](#).

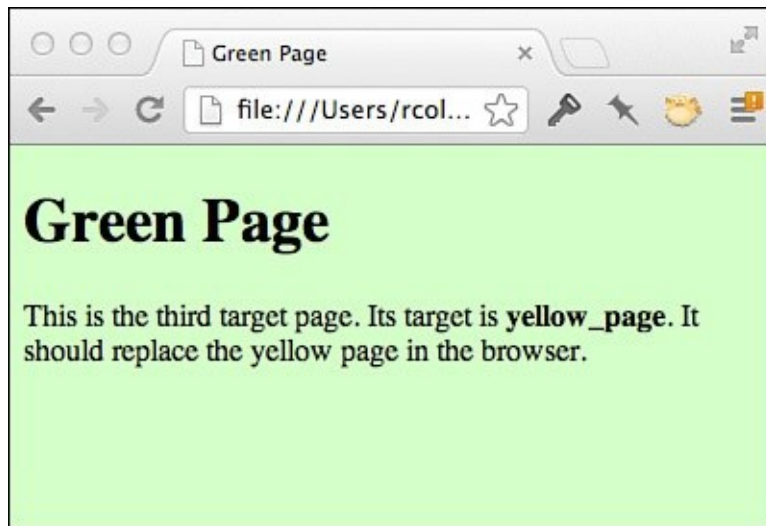


FIGURE 20.5 `green.html` displayed in the web browser window named `green_page`.

The `<base>` Tag

When you're using the `target` attribute with links, you'll sometimes find that all or most of the hyperlinks on a web page should point to the same browsing context. For example, you might want all the links on a page to open in a new window, so customers don't forget about your home page.

In such cases, instead of including a `target` attribute for each `<a>` tag, you can use another tag, `<base>`, to define a global target for all the links on a web page. The `<base>` tag is used as follows:

```
<base target="window_name">
```

If you include the `<base>` tag in the `<head> . . . </head>` block of a document, every `<a>` tag that doesn't have a `target` attribute will use the value of the `base` tag's `target` attribute as its default target. For example, if you had included the tag `<base target="yellow_page">` in the HTML source for `parent.html`, the three hyperlinks could have been written as follows:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
<head>
<title>Parent Window - Red</title>
<base target="yellow_page"> <!-- add base target="value" here -->
</head>
<body style="background-color: #ff9999">
  <h1>Parent Window - Red</h1>

  <p><a href="yellow.html">Open</a>
  <!-- no need to include a target -->
    the Yellow Page in a new window.</p>
  <p><a href="blue.html" target="blue_page">Open</a> the Blue Page in a new
window.</p>
  <p><a href="green.html">Replace</a>
  <!-- no need to include a target -->
```

```
        the yellow page with the Green Page.</p>
</body>
</html>
```

In this case, `yellow.html` and `green.html` load into the default window assigned by the `<base>` tag (`yellow_page`); `blue.html` overrides the default by defining its own target window of `blue_page`.

You also can override the window assigned with the `<base>` tag by using one of two special window names. If you use `target="_blank"` in a hyperlink, it opens a new browser window that doesn't have a name associated with it. Alternatively, if you use `target="_self"`, the current window is used rather than the one defined by the `<base>` tag.

Note

If you don't provide a `target` using the `<base>` tag and you don't indicate a target in a link's `<a>` tag, the link will load the new document in the same window as the link.

Inline Frames

The main advantage of inline frames is that you can position them anywhere on a web page, just as you can other elements like images or movies. You can incorporate content from another page or even another site into a page in a seamless way through the use of inline frames. In fact, inline frames, which are specified using the `<iframe>` tag, are very commonly the means by which “widgets” offered by popular websites are incorporated into other websites. For example, sites like Twitter and Facebook offer widgets that you can incorporate into your own site that are implemented using inline frames. The embed code that YouTube provides uses an `iframe` for the movie.

Here's a brief run-through of how to create inline frames. First, you define them using the `<iframe>` tag. Like images, these frames appear within the normal flow of an HTML document (hence the `i` for inline). The `<iframe>` tag supports attributes like `src`, which contains the URL of the document to be displayed in the frame, and `height` and `width`, which control the size of the frame.

[Table 20.2](#) lists the attributes of the `<iframe>` element.

Attribute	Description
width	Specifies the width, in pixels, of the floating frame that will hold the HTML document.
height	Specifies the height, in pixels, of the floating frame that will hold the HTML document.
src	Specifies the URL of the HTML document to be displayed in the frame.
srcdoc	Specifies HTML content to be displayed in the frame. This is new in HTML5.
name	Specifies the name of the frame for the purpose of linking and targeting.
sandbox	Enables extra restrictions for the content of an iframe. This is new in HTML5.

TABLE 20.2 Key Attributes

Because you know how to use inline images, using the `<iframe>` tag is fairly easy. The following code displays one way to use the Away from My Desk pages in conjunction with an inline frame. In this example, you begin by creating a page with a red background. The links that the user clicks appear on a single line, centered above the iframe. For clarity, I've placed each of the links on a separate line of code.

Following the links (which target the inline frame named `reason`), the code for the frame appears within a centered `<div>` element. As you can see in the following HTML, the floating frame will be centered on the page and will measure 450 pixels wide by 105 pixels high:

Input ▼

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
  <head>
    <title>I'm Away From My Desk</title>
    <style>
      body { background-color: #ffcc99; }
    </style>
  </head>
  <body>
    <h1>I'm away from my desk because ...</h1>
    <p style="text-align: center;">
      <a href="reason1.html" target="reason">Reason 1</a> |
      <a href="reason2.html" target="reason">Reason 2</a> |
      <a href="reason3.html" target="reason">Reason 3</a> |
      <a href="reason4.html" target="reason">Reason 4</a> |
      <a href="reason5.html" target="reason">Reason 5</a> |
      <a href="reason6.html" target="reason">Reason 6</a>
    </p>

    <div style="margin: 0 auto; width: 450px;">
      <iframe name="reason" src="reason1.html" style="width: 450px;
height:
105px;"></iframe>
    </div>
```



```
</body>
</html>
```

[Figure 20.6](#) shows the result.

Output ▼

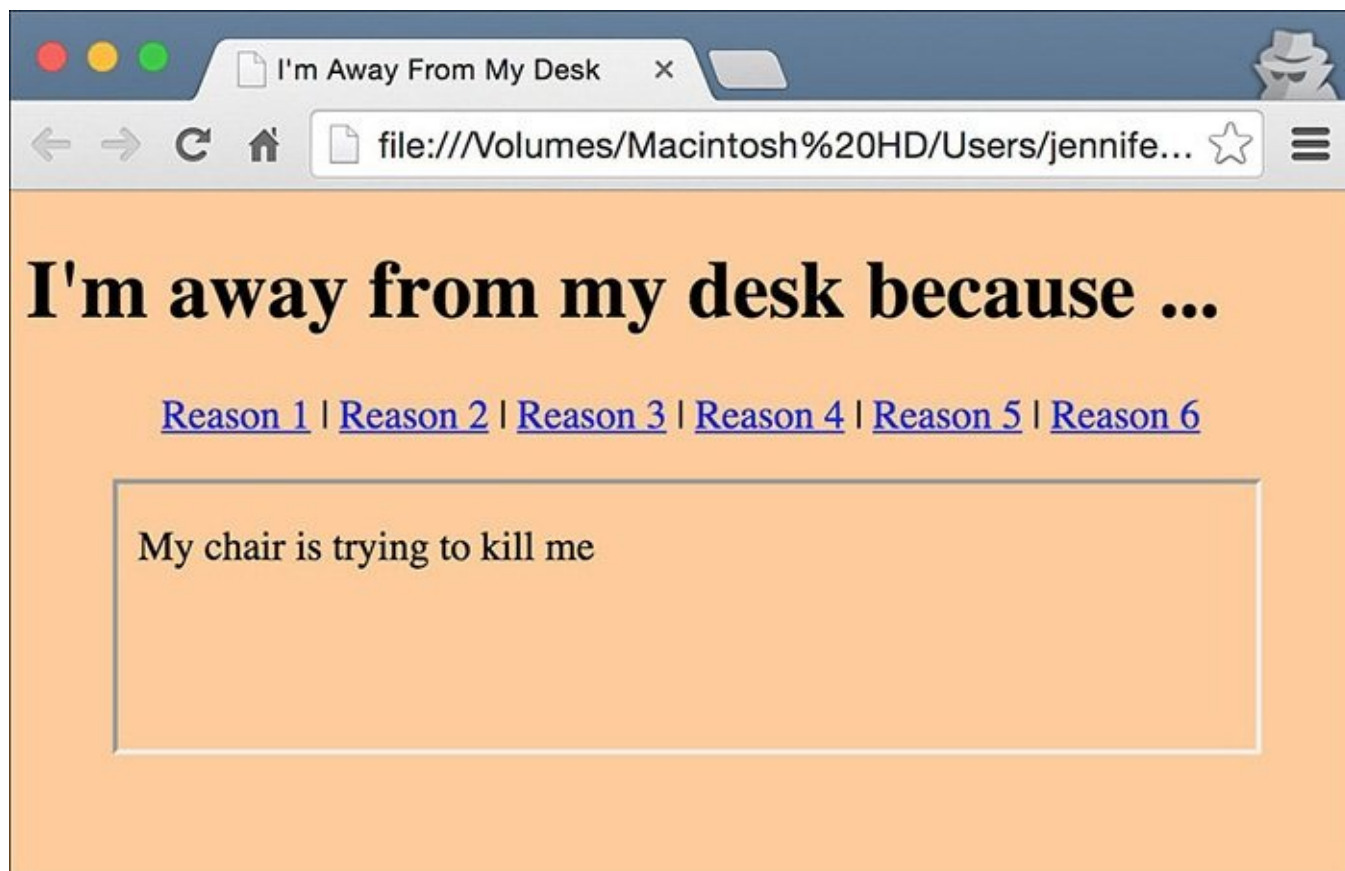


FIGURE 20.6 An inline frame.

You then create the reasons in separate HTML documents named `reason1.html`, `reason2.html`, `reason3.html`, and so on. These can be complete HTML documents with everything you might expect from a web page. But I made the reasons very simple. For example, here is the HTML for `reason2.html`:

[Click here to view code image](#)

```
<!doctype html>
<html>
  <head>
    <title>Reason 2</title>
  </head>
  <body>
    <p>I knew you were coming, so I'm baking a cake</p>
  </body>
</html>
```

This page is really simple and seems like a lot of work to change the text of the `iframe`. But with the new attribute `srcdoc`, you can place the HTML you want to display in the frame right in the `iframe` tag. Instead of referencing an entire page, the browser will load the HTML you specify. Here's how to set reason 1 as the default source document:

[Click here to view code image](#)

```
<iframe name="reason" src="reason1.html"
  srcdoc="<p>My chair is trying to kill me</p>"
  style="width: 450px; height: 105px"></iframe>
```

Note

You don't need a complete HTML document inside the `srcdoc` attribute. If the browser doesn't support the `srcdoc` attribute, however, it will show the HTML in the `src` attribute instead. So, don't leave that out. As of this writing, only Internet Explorer, Edge, and Opera Mini don't support the `srcdoc` attribute.

HTML5 also adds a security feature to iframes: the `sandbox` attribute. If you include this on your iframes, it will load them with extra restrictions. You can load your frames with all the restrictions by just including the attribute on your iframe tag like this:

[Click here to view code image](#)

```
<iframe src="frame.html" sandbox></iframe>
```

This will add the following security features to your framed content:

- The content will be treated as if it's coming from a foreign domain, even if it's not.
- Forms cannot be submitted from within the iframe.
- Scripts cannot be executed in the iframe.
- APIs are disabled.
- Links cannot target other browser contexts.
- Content cannot use embedded content, such as through the `<object>` or `<embed>` tags.
- Content cannot navigate to the top-level browsing context.
- Autoplay features such as video or form focus controls are blocked.

These features allow web designers to point to untrusted source material with less worry that a malicious script could be run on their web server. But the `sandbox` attribute also lets you open up the security for trusted sites with space-separated values of the `sandbox` attribute. These values are explained in [Table 20.3](#).

Attribute	Description
<code>allow-forms</code>	Enables form submission
<code>allow-pointer-lock</code>	Enables APIs
<code>allow-popups</code>	Enables pop-ups
<code>allow-same-origin</code>	Allows the iframe content to be treated as being from the same origin
<code>allow-scripts</code>	Enables scripts
<code>allow-top-navigation</code>	Allows the iframe content to navigate its top-level browsing context

TABLE 20.3 Sandbox Values

If you use inline frames to point to pages on websites you don't control, you should always sandbox them as much as possible. This will keep both your website and your customers more secure.

Opening Linked Windows with JavaScript

Pop-up windows are used all over the Web. They are often used to display advertisements, but they can be used for all sorts of other things as well, such as creating a separate window to show help text in an application or to display a larger version of a graph that's embedded in a document. You've seen how you can use the `target` attribute to open a link in a new window, but that approach isn't very flexible. You can't control the size of the window being displayed, nor which browser window controls are displayed.

Fortunately, with JavaScript you can take more control of the process of creating new windows. You've already learned that one of the objects supported by JavaScript is `window`. It refers to the window that's executing the script. To open a new window, you use the `open` method of the `window` object. Here's a JavaScript function that opens a window:

[Click here to view code image](#)

```
function popup(url) {
    mywindow = window.open(url, 'name', 'height=200,width=400');
    return false;
}
```

The function accepts the URL for the document to be displayed in the new window as an argument. It creates a new window using the `window.open` function and assigns that new window to a variable named `mywindow`. (I explain why we assign the new window to a variable in a bit.)

The three arguments to the function are the URL to be displayed in the window, the name for the window, and a list of settings for the window. In this case, I indicate that I want the window to be 400 pixels wide and 200 pixels tall. The name is important because if other links target a window with the same name, either via the `window.open()` function or the `target` attribute, they'll appear in that window.

At the end of the function, I return `false`. That's necessary so that the event handler used to call the function is stopped. To illustrate what I mean, it's necessary to explain how this function is called. Instead of using the `target` attribute in the `<a>` tag, the `onclick` handler can be used, as follows:

[Click here to view code image](#)

```
<a href="whatever.html" target="_blank" onclick="popup('whatever.html')">Pop  
up  
</a>
```

Of course, it's preferable to use unobtrusive JavaScript and add an appropriate event handler in a `script` tag. However, this works for the purposes of an example.

Ordinarily, when a user clicks the link, the browser calls the function and then goes right back to whatever it was doing before, navigating to the document specified in the `href` attribute. Returning `false` in the `popup()` function tells the browser not to continue what it was doing, so the new window is opened by the function, and the browser doesn't follow the link. If a user who had JavaScript turned off visited the page, the link to `whatever.html` would still open in a new window because I included the `target` attribute, too.

In the preceding example, I specified the `height` and `width` settings for the new window. There are several more options available as well, which are listed in [Table 20.4](#).

Setting	Purpose
<code>height</code>	Height of the window in pixels.
<code>width</code>	Width of the window in pixels.
<code>resizable</code>	Enable window resizing.
<code>scrollbars</code>	Display scrollbars.
<code>status</code>	Display the browser status bar.
<code>toolbar</code>	Display the browser toolbar.
<code>location</code>	Display the browser's location bar.
<code>menubar</code>	Display the browser's menu bar (not applicable on Mac OS X).
<code>left</code>	Left coordinate of the new window onscreen (in pixels). By default, pop-up windows are placed slightly to the right of the spawning window.
<code>top</code>	Top coordinate of the new window onscreen (in pixels). By default, pop-up windows are placed slightly below the top of the spawning window.

TABLE 20.4 Settings for Pop-Up Windows

When you specify the settings for a window, you must include them in a comma-separated list, with no spaces anywhere. For the settings that allow you to enable or disable a browser interface component, the valid values are `on` or `off`. Here's a valid list of settings:

[Click here to view code image](#)

```
status=off,toolbar=off,location=off,left=200,top=100,width=300,height=300
```

Here's an invalid list of settings:

[Click here to view code image](#)

```
status=off, toolbar=off, location=false, top=100
```

Including spaces (or carriage returns) anywhere in your list will cause problems. It's also worth noting that when you provide settings for a new window, the browser automatically assumes a default of `off` for any on/off settings that you don't include. So, you can leave out anything you want to turn off.

Here's a complete example that uses JavaScript to create a new window:

[Click here to view code image](#)

```
<!DOCTYPE html>
<html>
  <head>
    <title>Popup example</title>
  </head>
  <body>
    <h1>Popup Example</h1>
    <p>
      <a href="popup.html" onclick="popup(this.href)">Launch popup</a>
    </p>
    <script>
      function popup(url) {
        var mywindow = window.open(url, 'name', 'height=200,width=400');
        return false;
      }
    </script>
  </body>
</html>
```

When a user clicks the Launch pop-up link, a new 200×400 pixel window appears with the contents of `popup.html`.

The unobtrusive approach is to skip the `onclick` attribute entirely and bind the `popup()` function to the link in your JavaScript code. First, change the link on the page to look like this:

[Click here to view code image](#)

```
<a href="popup.html" id="launchpopup">Launch popup</a>
```

Then you should edit the `<script>` tag so that it looks like this:

[Click here to view code image](#)

```
<script>
  function popup(url) {
    var mywindow = window.open(url, 'name', 'height=200,width=400');
    return false;
  }
  window.onload = function () {
    var link = document.getElementById("launchpopup");
    link.onclick = function () {
      return popup(this.href);
    }
  }
</script>
```

In this case, when the page loads, I retrieve the link by its ID and then bind a new anonymous function to it that calls the original `popup()` function. Instead of hard coding the URL, I pass `this.href` to the `popup()` function so that it opens the URL in the link.

Using a library like jQuery can make things even easier. Suppose you want any link tag with the class `popup` to open a new window with the URL associated with the link.

Here's the code:

[Click here to view code image](#)

```
<script>
$(document).ready(function () {
    $("a.popup").click(function (event) {
        var mywindow = window.open(this.href, 'newwindow',
        'height=200,width=400');
        event.preventDefault();
    });
});
</script>
```

When the page is ready, I apply the same `onclick` handler to all the links on the page with the class `popup`. The anonymous event handler opens a new window with the URL stored in `this.href`, which returns the URL in the link that the user clicked. It then calls the `preventDefault()` method on the event. It's used instead of just returning `false` because it doesn't disrupt other event handlers that may be fired in addition to this one.

Summary

In this lesson, you learned how to link a document to a new or existing window. In addition, you learned how to create inline frames and link them by using the tags listed in [Table 20.5](#).

Tag	Attribute	Description
<code><base target="window"></code>		Sets the global link window for a document.
<code><iframe></code>		Defines an inline frame.
	<code>src</code>	Indicates the URL of the document to be displayed in the frame.
	<code>srcdoc</code>	Provides the HTML source to be displayed in the frame.
	<code>name</code>	Indicates the name of the frame for the purpose of linking and targeting.
	<code>width</code>	Indicates the width of the frame in pixels.
	<code>height</code>	Indicates the height of the frame in pixels.
	<code>sandbox</code>	Enables extra restrictions for the content of the frame. Possible values include <code>allow-forms</code> , <code>allow-pointer-lock</code> , <code>allow-popups</code> , <code>allow-same-origin</code> , <code>allow-scripts</code> , and <code>allow-top-navigation</code> . If none are listed, all restrictions apply.

TABLE 20.5 New Tags Discussed in [Lesson 20](#)

Workshop

As if you haven't had enough already, here's a refresher course of questions, quizzes, and exercises that will help you remember some of the most important points you learned in this lesson.

Q&A

Q Is there any reason why you should sandbox iframe content from your own server?

A The decision to sandbox content shouldn't be made based on where the content is coming from, but rather how reliable and trustworthy you find the source. If you are posting your pages to a shared domain, you might have no knowledge of the designers of another site on that domain. Sandboxing helps you maintain security.

Q What if I use the `<frame>` tag anyway?

A Most browsers still support frames, so your content will still work. However, it is considered poor form. And with iframes and CSS, there are many options that don't have the drawbacks of frames while providing the same functionality.

Quiz

1. Where does the keyword `_self` take readers if they click a link with that target?

2. If you want a link to open in a named iframe, what attribute do you need to use?
3. When a web page includes the `<frameset>` element, what element cannot be used at the beginning of the HTML document?
4. What attribute of the `<iframe>` tag provides security for the reader and the website loading the content?

Quiz Answers

1. The `_self` target is the default action for a link. The link will open in the same browser context as the link itself.
2. The `target` attribute of the `<a>` tag directs linked pages to load into the appropriate iframe.
3. When a web page includes the `<frameset>` element, it cannot include the `<body>` element at the beginning of the page. They're mutually exclusive. And this is why frames are no longer part of HTML5.
4. The `sandbox` attribute provides some security for the reader and the website by putting restrictions on what the iframe content can do.

Exercises

1. Create an iframe that links to another page on your website. Size it so that it fits in the main content area of your page.
2. For the preceding frameset, create a page that you will use for a table of contents beside the frame. Create two links in the table of contents that open in the inline frame.